Boolean Algebra & Logic Circuits

MASS. INST. TECH.
DEC 20 1940
LIBRARY

A SYMBOLIC ANALYSIS

OF

RELAY AND SWITCHING CIRCUITS

by

Claude Elwood Shannon

B.S., University of Michigan

1936

Submitted in Partial Fulfillment of the

Requirements for the Degree of

MASTER OF SCIENCE

from the

Massachusetts Institute of Technology

1940

Signature of Author_____

Department of Electrical Engineering, August 10, 1937

Signature of Professor
in Charge of Research_____

Signature of Chairman of Department
Committee on Graduate Students_____

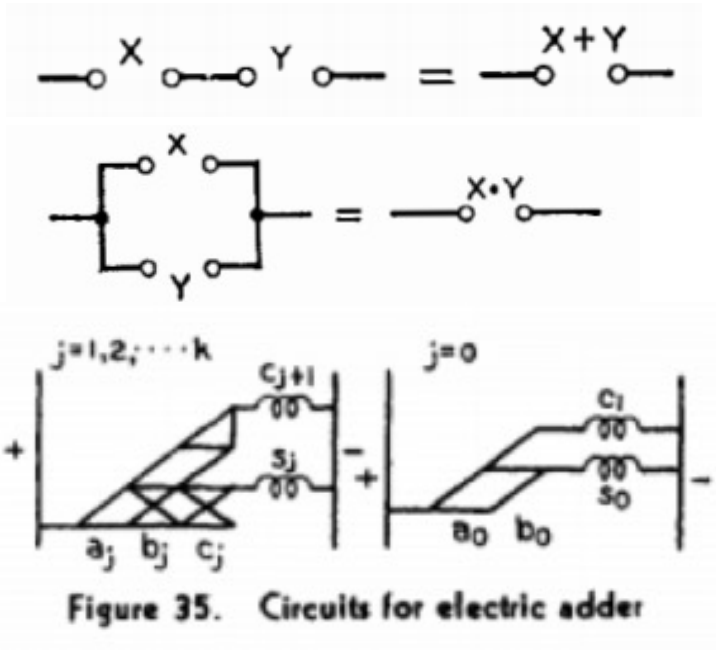


Figure 35. Circuits for electric adder

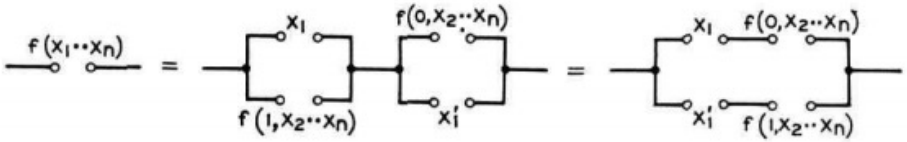


Figure 4. Expansion about one variable

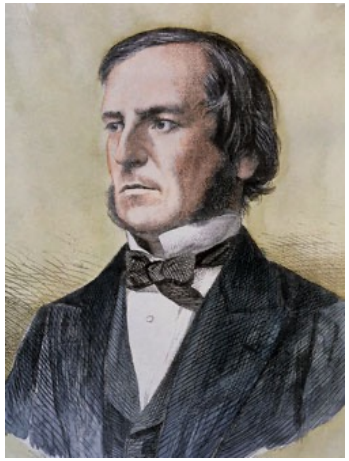
Chee Wei Tan

Outline

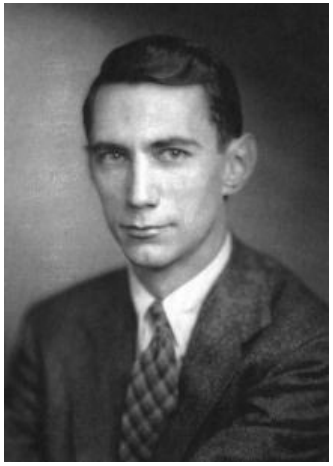
- **Basics of Combinational Circuits**
  - Boolean Algebra
  - Truth Table & Logic Gates
- **Canonical Implementation**
  - Sum-of-Products (SOP)
  - Product-of-Sum (POS)
- **Circuit Simplification**
  - Karnaugh Map
  - Half Adder and Full Adder
- **Designing Large Logic Circuits**
  - Shannon's Expansion Theorem
  - Topological ordering and Hardware Description Program
- **Logic Games**
  - Logic Diagrams and Puzzles

Background Reading: Chapters 4 and 5 in Textbook

Boolean Algebra – Fundamental Basis of Computer

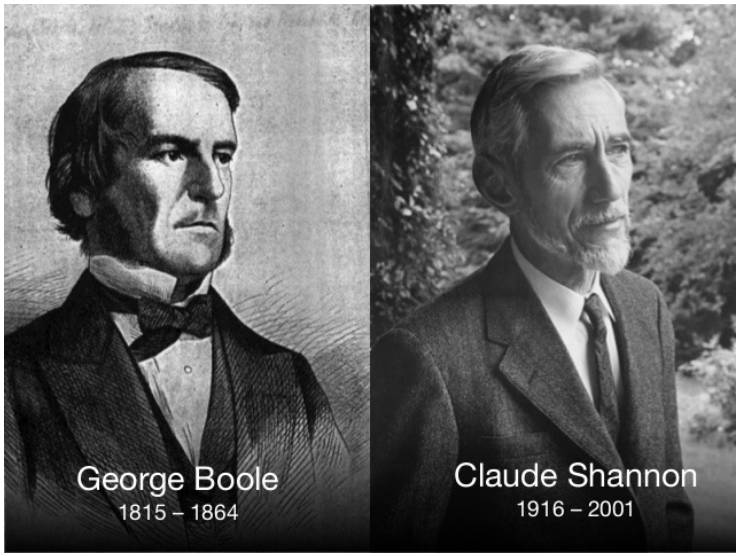


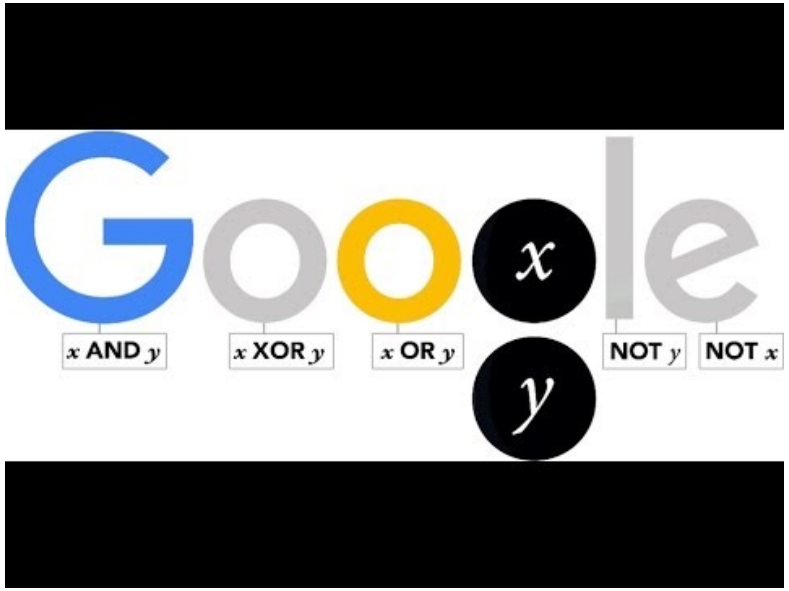
- **George Boole**
  - 1815-1864
  - English mathematician, philosopher, logician
  - Inventor of Boolean logic
  - Symbolic logic

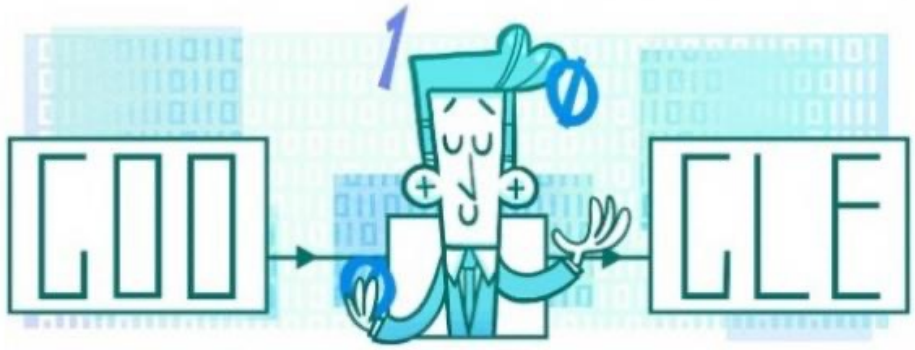


- **Claude Shannon**
  - 1916-2001
  - American mathematician and engineer
  - Father of Information Theory
  - His thesis at MIT was viewed as *possibly the most important, and also the most famous master's thesis of the century.* Introduce **Switching Algebra** to analyze and design physical circuits algebraically at the age of 21 in 1937
  - Pioneer of digital information age

Boolean Algebra – Fundamental Basis of Computer



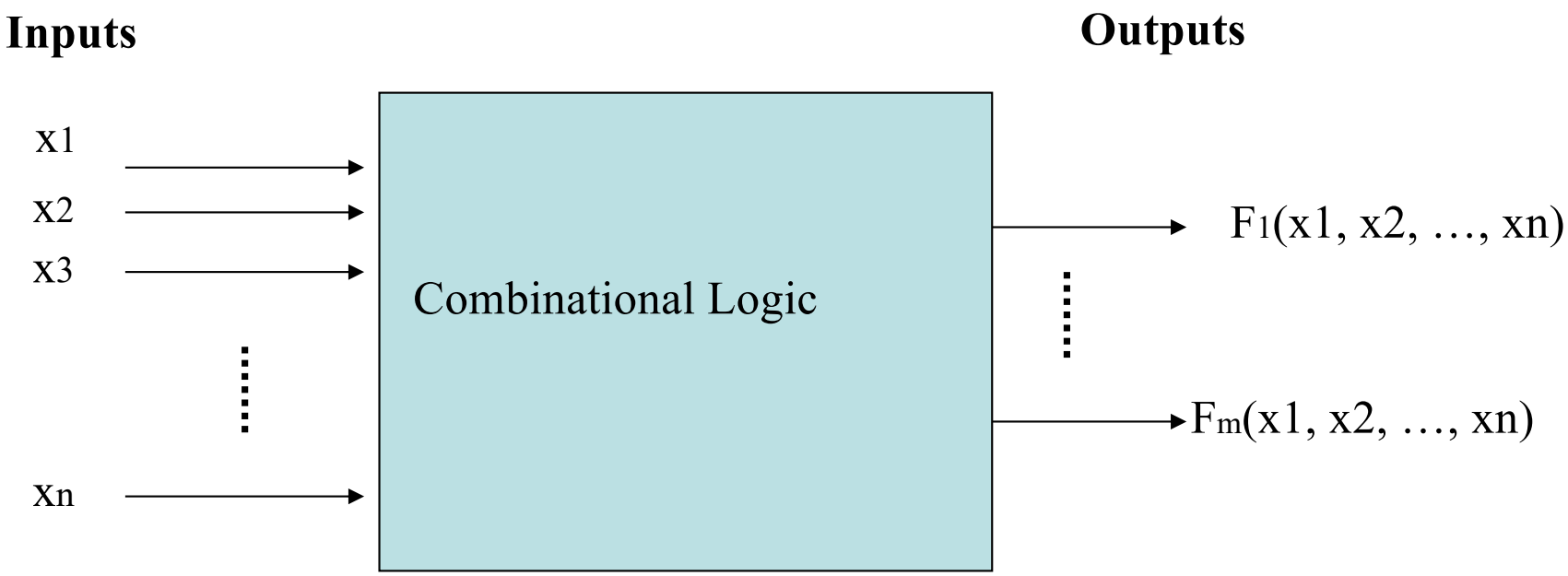




https://www.google.com/doodles/george-booles-200th-birthday

https://www.google.com/doodles/claude-shannons-100th-birthday

Combinational Logic



Combinational logic circuit: a logic circuit in which logical decisions are made based only on the combinations of the inputs.
Each input or each output takes only two possible values (true or false).

The outputs are updated immediately after all the inputs are read according to the mapping functions $F_1(x1, x2, \ldots, xn)$, …, $F_m(x1, x2, \ldots, xn)$.

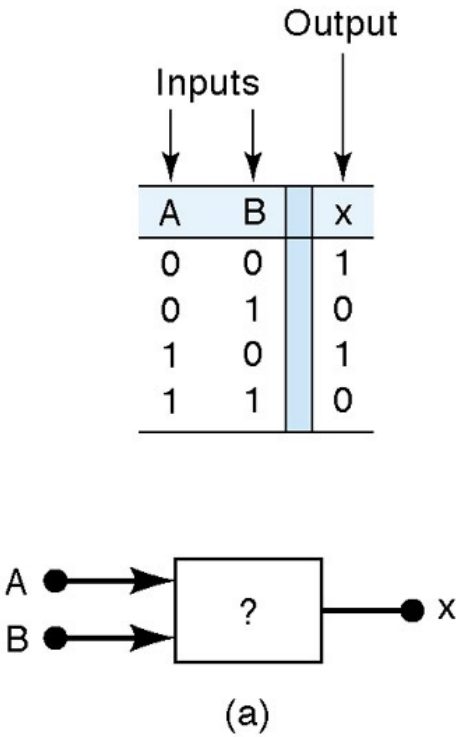
Truth Table

- Each mapping function is described by a **Truth Table**.

Inputs		Output
A	*B*	*Z*
0	0	1
0	1	0
1	0	0
1	1	1

- Truth table is like a “Blueprint” for design and describes the relationship between the input and output of a logic circuit.
- It records only two possible values using 0 or 1 to represent False and True respectively. The number of entries corresponds to the number of inputs. For example a 2 input table would have $2^2 = 4$ entries. A 3 input table would have $2^3 = 8$ entries.

Mapping Functions and Truth Table



A	B	x
0	0	1
0	1	0
1	0	1
1	1	0

A	B	C	x
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

(b)

A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

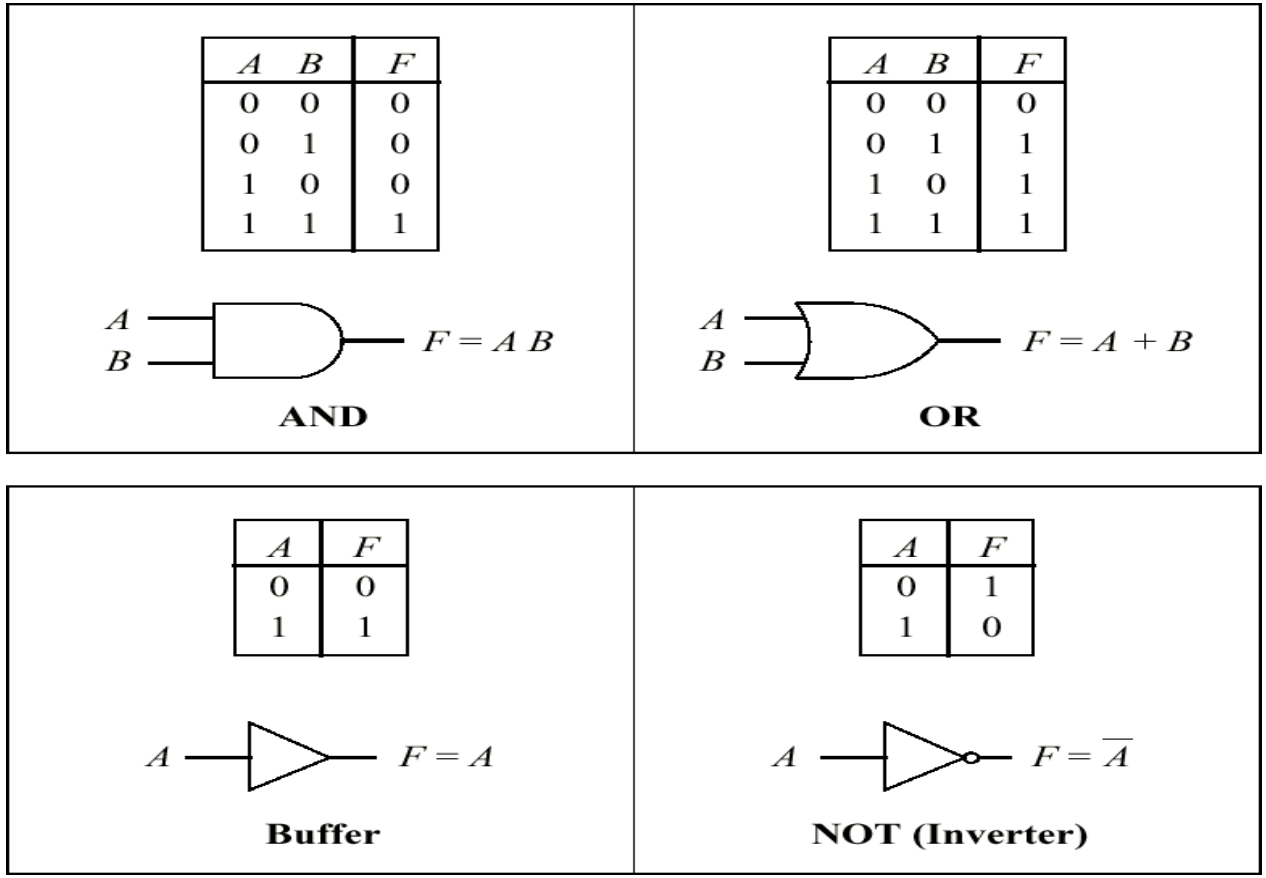
(c)

- Given the inputs, we want to design each mapping function in the combinational logic circuit (a) to get the outputs indicated on the truth table.
- **Method of Perfect induction**: Truth table can also be used to prove any Boolean statement using proof by exhaustion.

Logic Gates and Their Symbols

Logical symbols for AND, OR, and NOT Boolean functions.

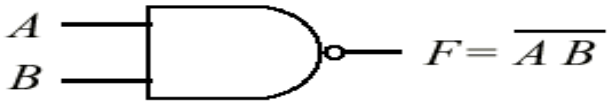
Buffer simply means equivalence mathematically (but interpret as "holding area" in circuits)



- Note the use of the "inversion bubble."
- Be careful about the "nose" of the gate when drawing AND vs. OR.

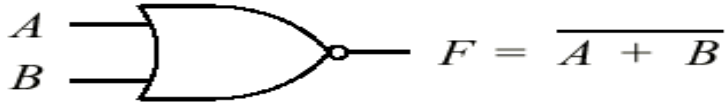
Logic symbols for NAND, NOR, XOR, and XNOR Boolean functions

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0



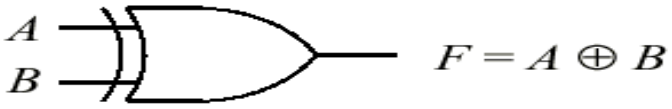
NAND

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0



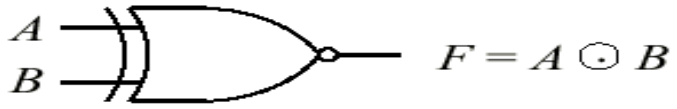
NOR

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



Exclusive-OR (XOR)

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

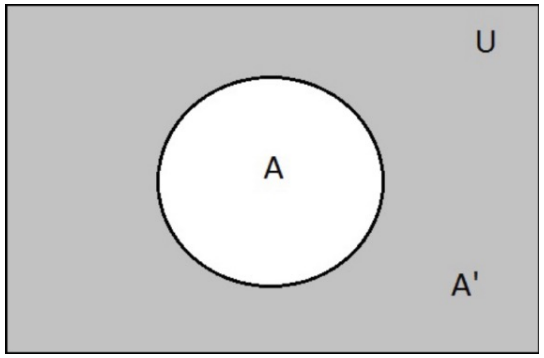


Exclusive-NOR (XNOR)

Special Case of Single Input (Tetralemma)

Given n inputs, we have a total of 2^{2^n} functions.

- Tetralemma (Tetra and lemma means four and small theorem respectively):
- It states that with reference to any a logical proposition X, there are four possibilities.



- Example: The world is finite.
  The world is infinite.
  The world is both finite and infinite.
  The world is neither finite nor infinite.
- Indian Logic – Chapter 3 by Amita Chatterjee, Leila Haaparanta (editor), The Development of Modern Logic, Oxford University Press, page 927.

Truth Tables For All Possible Functions with Two Inputs (Binary Variables)

Inputs		Outputs							
A	B	*False*	*AND*	$A\overline{B}$	A	$\overline{A}B$	B	*XOR*	*OR*
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

Given 2 inputs, we have totally

$$2^{2^2} = 2^4 = 16$$

functions.

Inputs		Outputs							
A	B	*NOR*	*XNOR*	$\overline{B}$	$A + \overline{B}$	$\overline{A}$	$\overline{A} + B$	*NAND*	*True*
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

Given n inputs, we have a total of

2^{2^n} functions.

- The frequently-used functions have special names: AND, XOR, OR, NOR, XOR, and NAND (Always use upper case spelling)
- How many of these functions involve both inputs?

Basics of Boolean Algebra

	Relationship	Dual	Property
Postulates	$A\,B = B\,A$	$A + B = B + A$	Commutative
	$A\,(B + C) = A\,B + A\,C$	$A + B\,C = (A + B)\,(A + C)$	Distributive
	$1\,A = A$	$0 + A = A$	Identity
	$A\,\overline{A} = 0$	$A + \overline{A} = 1$	Complement
Theorems	$0\,A = 0$	$1 + A = 1$	Zero and one theorems
	$A\,A = A$	$A + A = A$	Idempotence
	$A\,(B\,C) = (A\,B)\,C$	$A + (B + C) = (A + B) + C$	Associative
	$\overline{\overline{A}} = A$		Involution
	$\overline{A\,B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}\,\overline{B}$	DeMorgan's Theorem
	$AB + \overline{A}C + BC = AB + \overline{A}C$	$(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)$	Consensus Theorem
	$A\,(A + B) = A$	$A + A\,B = A$	Absorption Theorem

A, B, etc. are called Boolean variables or Literals.

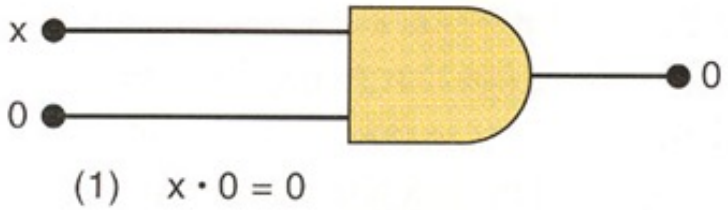
The values 0 and 1 are called constants.

Principle of Duality:

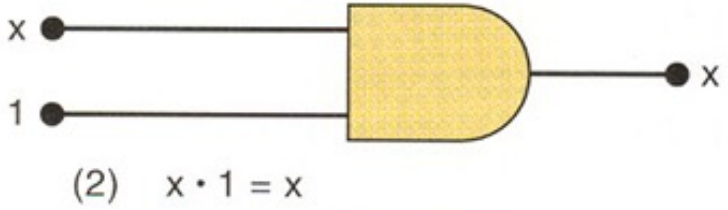
The **dual** of a Boolean function statement is obtained by:

- replace AND with OR;
- replace OR with AND,
- replace 1's by 0's,
- replace 0's by 1's

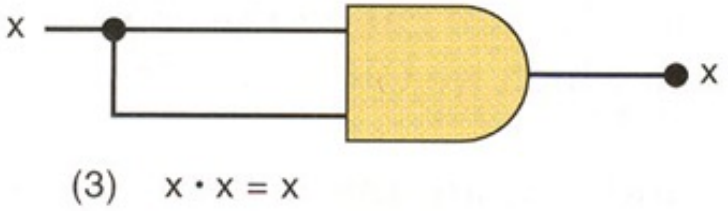
Single-Variable Boolean Theorems



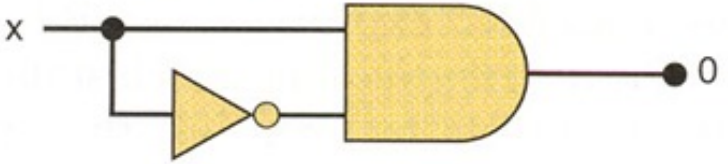
(1) $x \cdot 0 = 0$



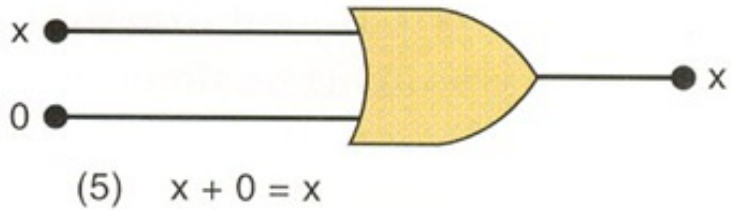
(2) $x \cdot 1 = x$



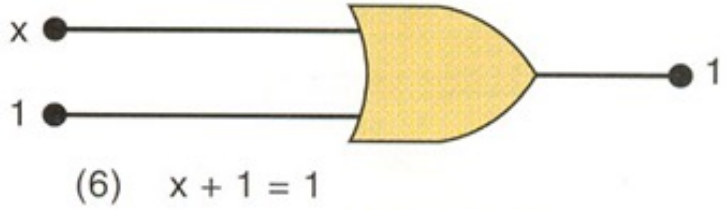
(3) $x \cdot x = x$



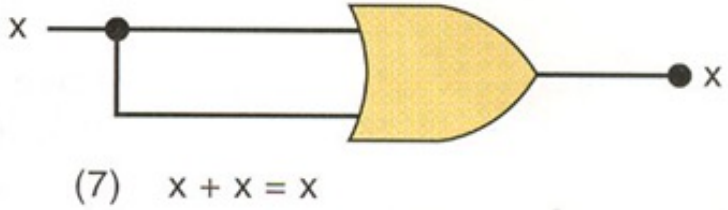
(4) $x \cdot \bar{x} = 0$



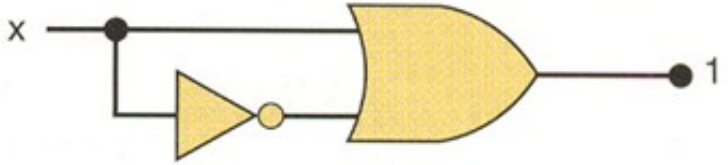
(5) $x + 0 = x$



(6) $x + 1 = 1$

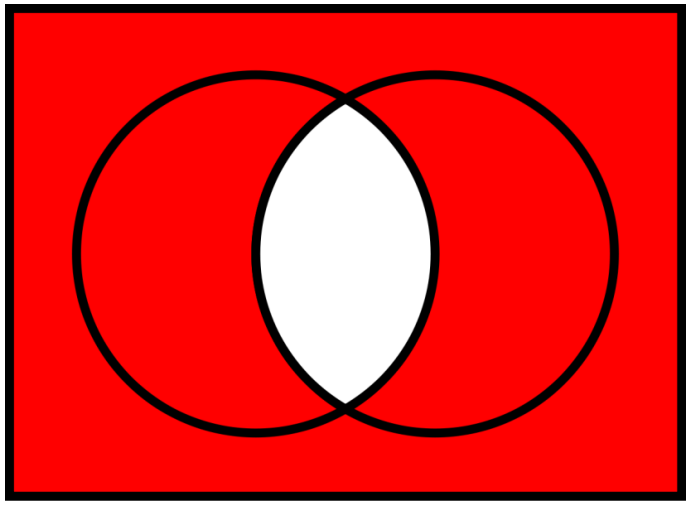


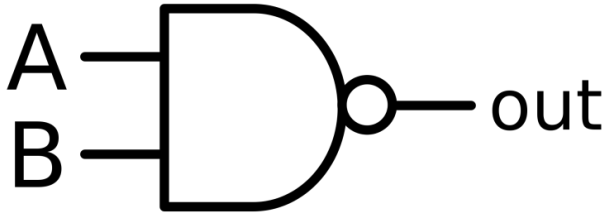
(7) $x + x = x$



(8) $x + \bar{x} = 1$

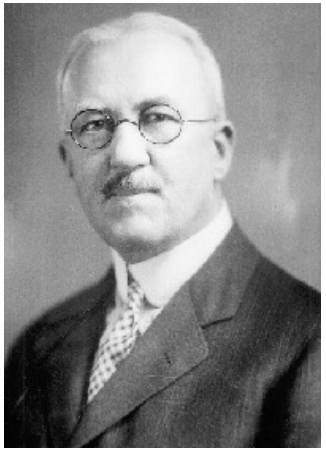
Sheffer Stroke (or NAND Gate)







Henry Maurice Sheffer, American logician coined "**Boolean Algebra**"

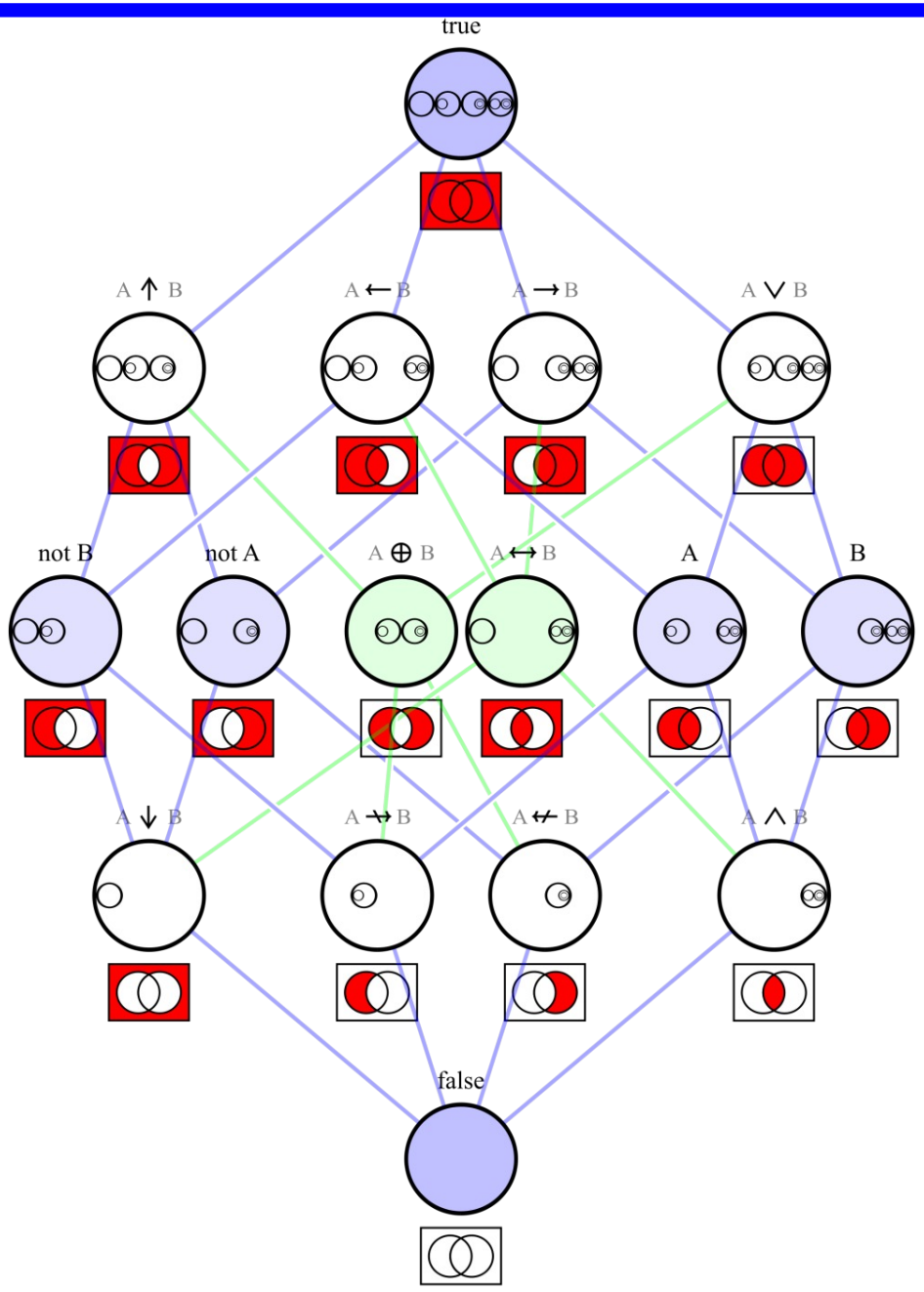


- Universal property of NAND gate: can realize AND, OR, NOT
- Can you prove this?
- The dual of the NAND gate is the NOR gate
- Both NAND and NOR gates are know as universal gates
- The Apollo Guidance Computer on Apollo spaceship used entirely NOR gates in their electronic circuits:

https://www.hq.nasa.gov/alsj/ic-pg3.html

Edward Huntington, American mathematician, and the **Huntington's Axiom** $(a'+b')'+(a'+b)' = a$

Hasse Diagram of the two-input Boolean Algebra



DeMorgan's Theorem

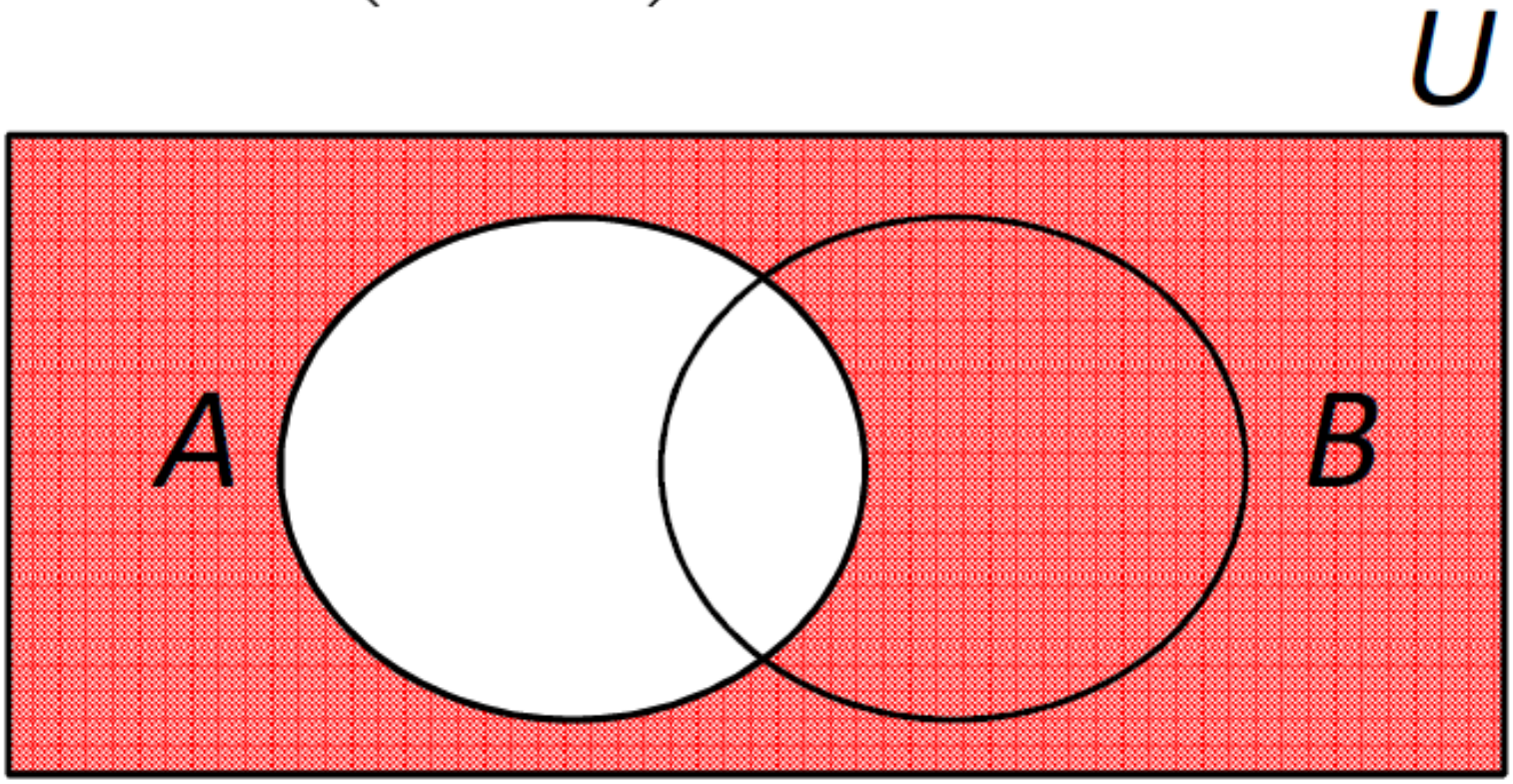


A B	$\overline{A\,B} = \overline{A} + \overline{B}$		$\overline{A + B} = \overline{A}\,\overline{B}$	
0 0	1	1	1	1
0 1	1	1	0	0
1 0	1	1	0	0
1 1	0	0	0	0

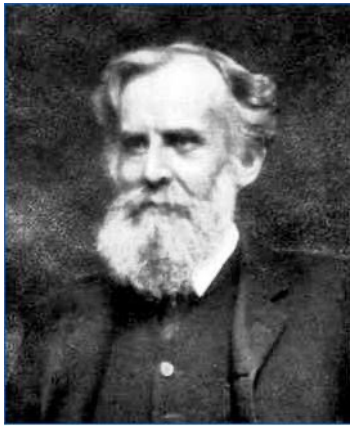
DeMorgan's theorem: $A + B = \overline{\overline{A + B}} = \overline{\overline{A}\,\overline{B}}$

DeMorgan's Theorem (Proof by Logic Diagram)

$$(A \cup B)' = A' \cap B'$$



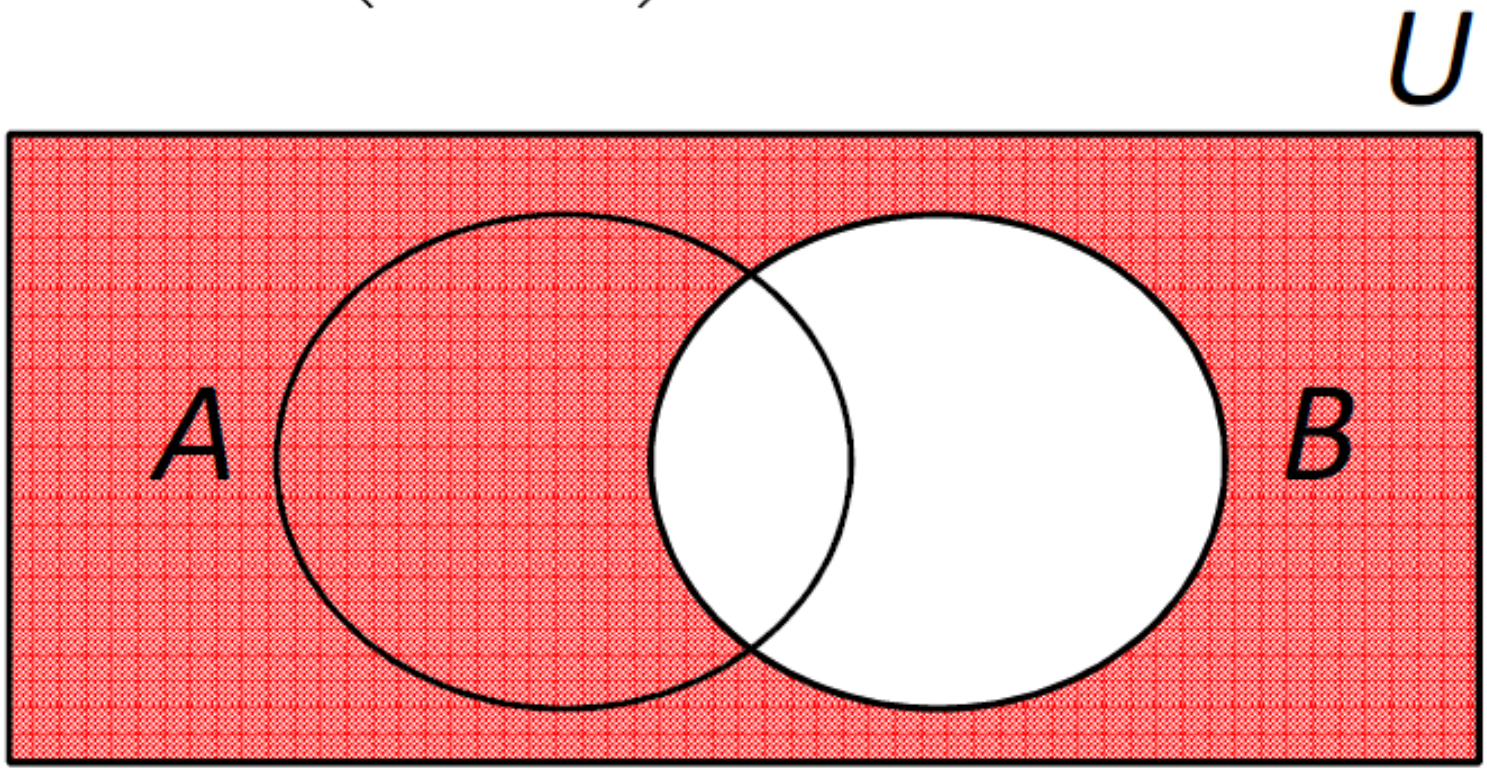
This is A'.



John Venn
English logician
Inventor of
Venn's Diagram

DeMorgan's Theorem (Proof by Logic Diagram)

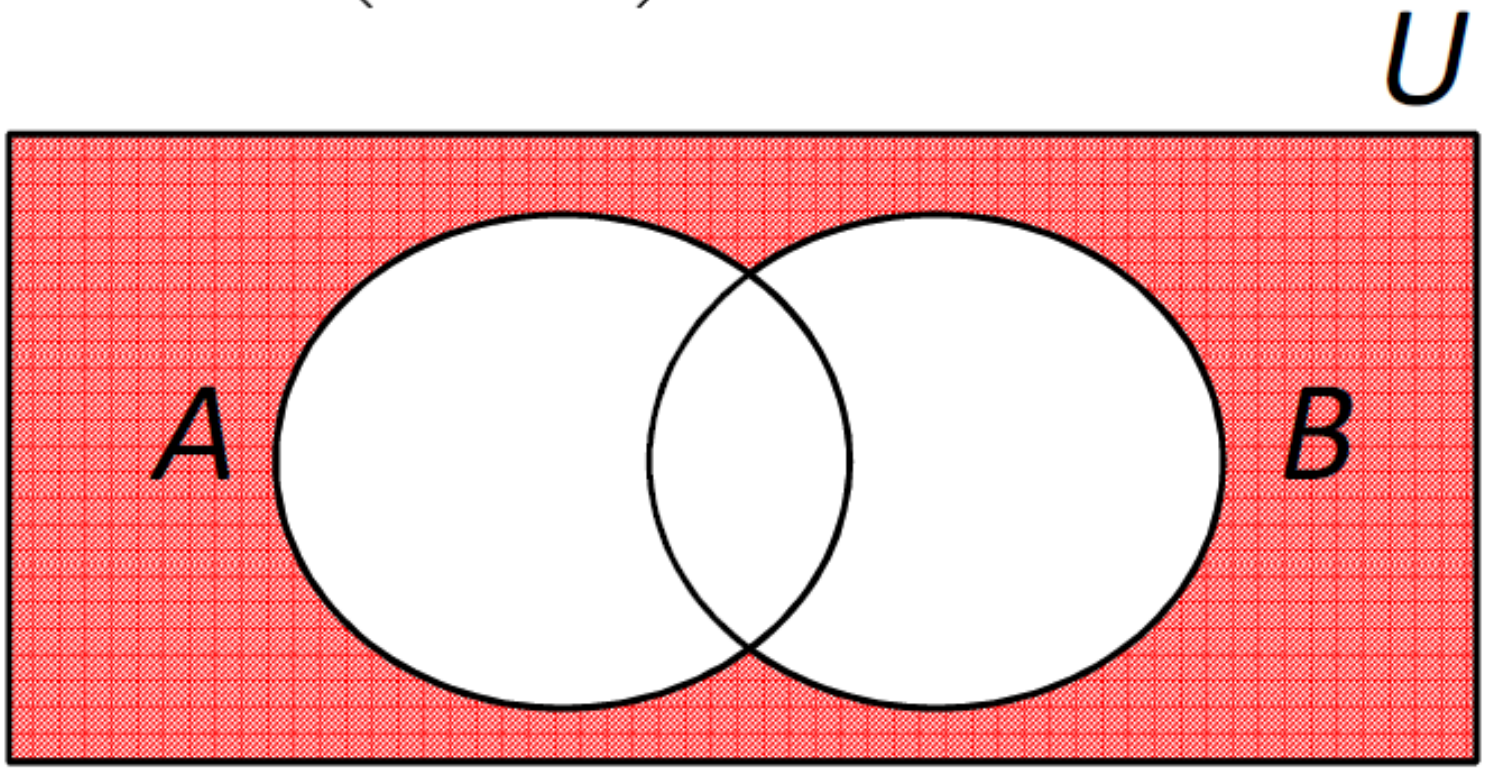
$$(A \cup B)' = A' \cap B'$$



This is B'.

DeMorgan's Theorem (Proof by Logic Diagram)

$$(A \cup B)' = A' \cap B'$$



This is their intersection.

DeMorgan's Theorem (Application)

- DeMorgan's Theorem can simplify Boolean expressions:
  1) Break a long bar over an expression.
  2) When the long bar is broken, the operation directly underneath the break changes from OR to AND, or vice versa.
  3) And the broken bar pieces remain over the individual variables
- Simple rules to follow:

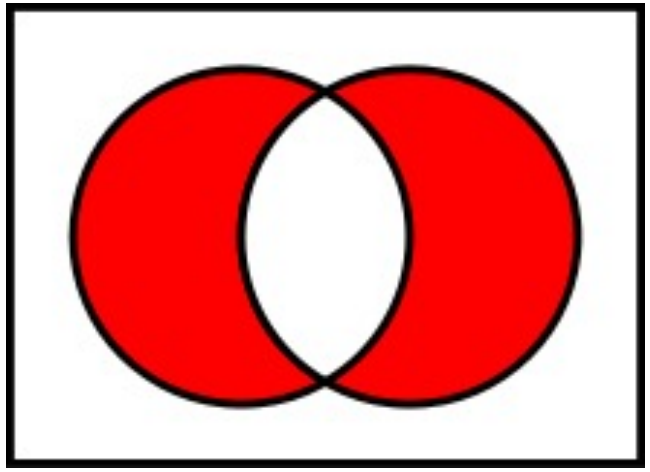
  Break one bar at a time - don't try to do it all in one line

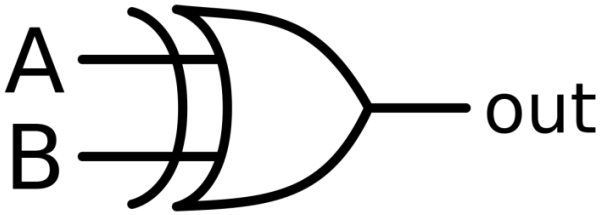
  Does not matter which bar you break first, so try the longest first.
- Intuitive understanding:

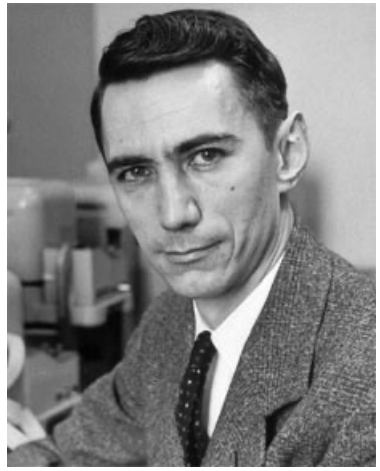
  "He cannot be both awake *and* sleeping."

  "He is *not* awake *or* he is *not* sleeping."
- Use DeMorgan's Theorem to simplify $\overline{A + \overline{B.C}}$

XOR Gate







Father of the **Bit**

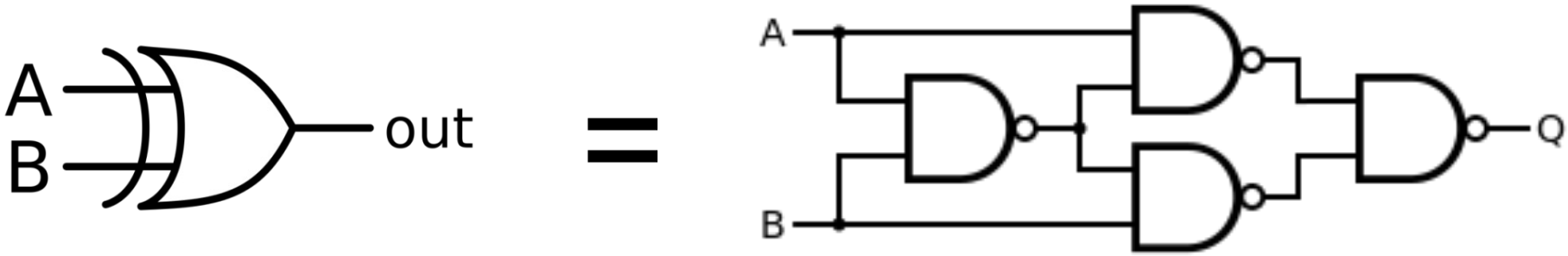
- True if exactly one but not both:
  - "This weekend, I am going to London, or I am going to Melbourne."
- The **bit-wise XOR** of two numbers in natural binary form is mathematically equivalent to **Addition modulo 2**
  Also known as the **Digital Sum of Base 2**

⊕	0	1
0	0	1
1	1	0

- Shannon's thesis demonstrates the first Combination Lock — an electrical circuit that works if, and only if, the right combination of buttons is pressed. How can XOR gate be used?

●https://www.nature.com/nature/journal/v410/n6830/full/410768a0.html ●https://youtu.be/nS0luYZd4fs

XOR Gate (Boolean algebraic analysis)



Method to represent XOR as four NAND gates:

```
A^B =       AB' +  A'B     // add forms of 0, BUF rule
    =   AA'+AB' +  A'B+BB' // factor out A and B, algebra
    =  A(A'+B') + (A'+B')B // represent (A'+B') as C :: Reminder (A'+B')=(AB)'
    =        AC + CB       // represent AC as X and CB as Y
    =         X + Y        // substitute  X+Y as (X'Y')', OR rule
    =         (X'Y')'      // restore X and Y to original values
    =      ((AC)'(CB)')'   // restore C as (AB)', equivalent to (A'+B')
    = ((A(AB)')'((AB)'B)')'// DONE, XOR as 4 NAND gates, sharing the (AB)' line
```

Another method to do the same:

```
A^B =       (A+B)(AB)'     // represent (AB)' as C
    =       (A+B)C         // Distribute C
    =         AC+BC        // represent AC as X and BC as Y
    =          X+Y         // substitute  X+Y as (X'Y')', OR rule
    =         (X'Y')'      // restore X and Y to original values
    =      ((AC)'(BC)')'   // restore C as (AB)', equivalent to (A'+B')
    = ((A(AB)')'(B(AB)')')'// DONE, XOR as 4 NAND gates, sharing the (AB)' line
```

https://hackernoon.com/xor-the-magical-bit-wise-operator-24d3012ed821

Outline

- **Basics of Combinational Circuits**
  - Boolean Algebra
  - Truth Table & Logic Gates
- **Canonical Implementation**
  - Sum-of-Products (SOP)
  - Product-of-Sum (POS)
- **Circuit Simplification**
  - Karnaugh Map
  - Half Adder and Full Adder
- **Designing Large Logic Circuits**
  - Shannon's Expansion Theorem
  - Topological ordering and Hardware Description Program
- **Logic Games**
  - Logic Diagrams and Puzzles

Background Reading: Chapters 4 and 5 in Textbook

The Sum-of-Products (SOP) Form

Minterm Index	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

What can this logic function be used for?

- Minterm is defined as the Boolean expression with minimum terms satisfying the output.
- F is true when:

  A=0 B=1 C=1 Minterm 3

  A=1 B=0 C=1 Minterm 5

  A=1 B=1 C=0 Minterm 6

  A=1 B=1 C=1 Minterm 7

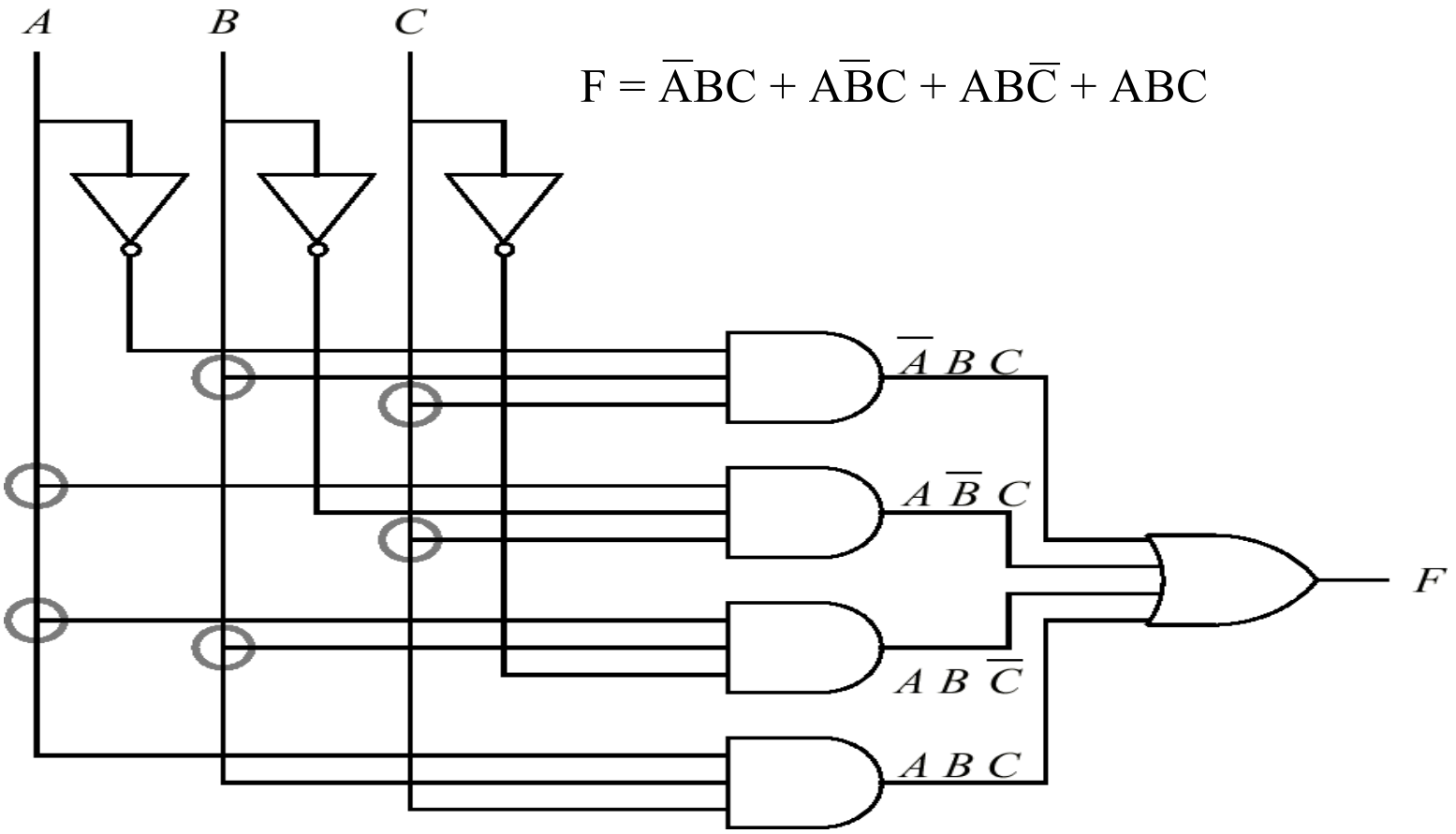
$$F = \Sigma\,(3, 5, 6, 7)$$

- $F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$

The SOP (Sum-of-Products) form is a two-level AND-OR equation.

A 2-Level AND-OR Circuit that Implements the Majority Function (The SOP Form)

- A SOP form is a two-level AND-OR equation
- Implement the SOP form with an arrangement of logic gates from the set {AND, OR, NOT}



The Product-of-Sum (POS) Form

Minterm Index	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

- Minterms: $F = \Sigma\,(3, 5, 6, 7)$
- SOP:

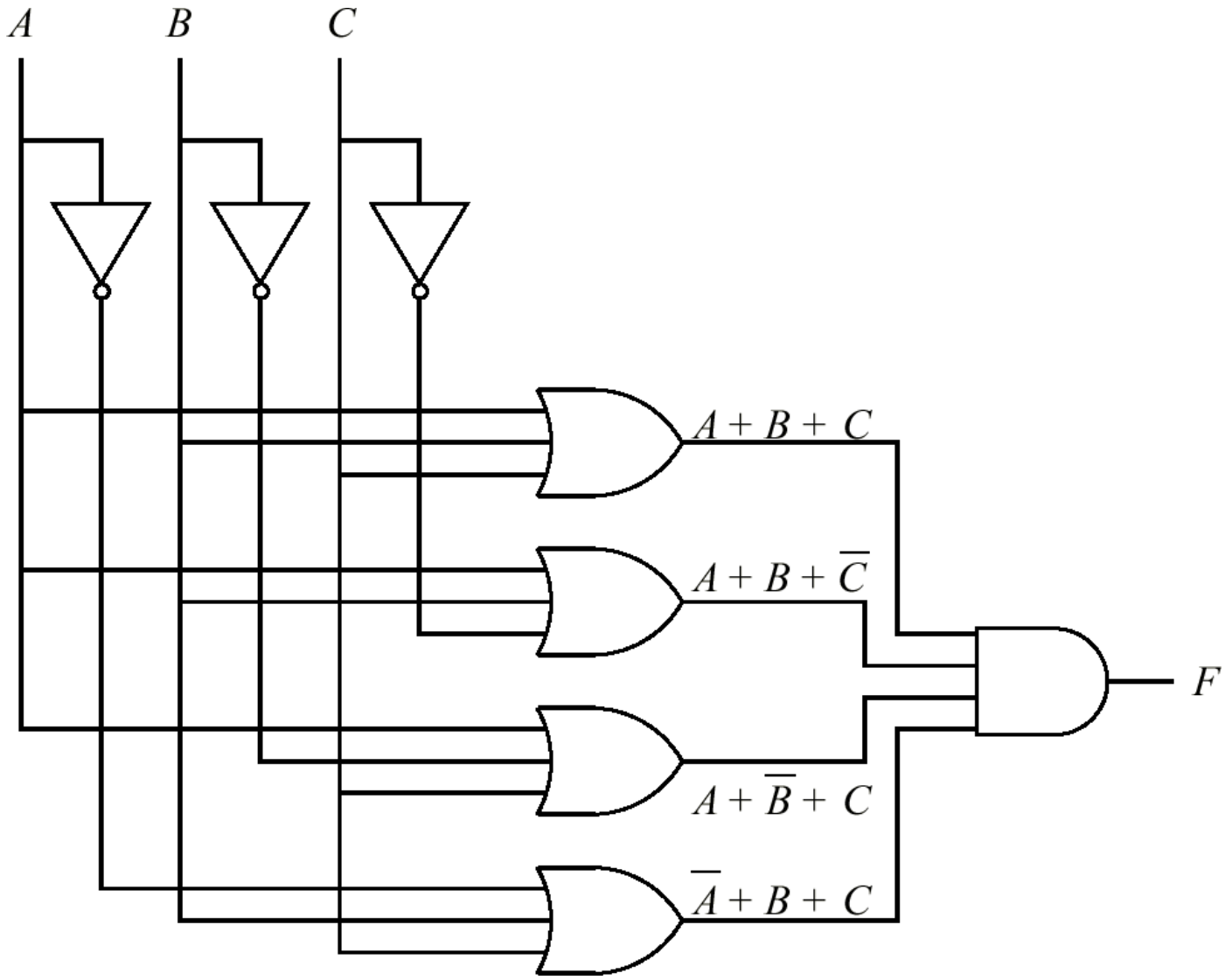
$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

- POS: Product-of-Sum

$$F = (\overline{\bar{A}B\bar{C}})\,(\overline{\bar{A}BC})\,(\overline{\bar{A}BC})(\overline{A\bar{B}C})$$
$$= (A+B+C)(A+B+\bar{C})(A+\bar{B}+C)(\bar{A}+B+C)$$

The Product-of-Sum form (POS) is a two-level OR-AND equation

A 2-Level OR-AND Circuit that Implements the Majority Function (The POS Form)



Design Ideas of Digital Circuits at Different Scale

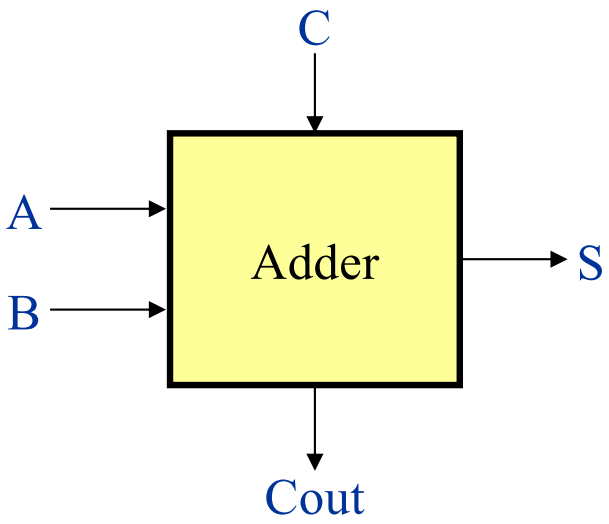
- Digital circuit designs are normally made using collections of logic gates referred to as transistor components and integrating them into a single chip.
- Levels of integration (numbers of gates) in an integrated circuit (IC):
  - Small scale integration (SSI): 1-10 logic gates.
    - Apollo Computer in Apollo program and Minuteman missile
  - Medium scale integration (MSI): 10 to 100 logic gates.
  - Large scale integration (LSI): 100,000 logic gates.
  - Very large scale integration (VLSI): 100,000-upward logic gates.
    - Apple iPhone, Xbox, etc
- Computational barrier as the number of logic gates scales up
  - Breaking the barrier requires deeper insights in mathematics and physics

Outline

- **Basics of Combinational Circuits**
  - Boolean Algebra
  - Truth Table & Logic Gates
- **Canonical Implementation**
  - Sum-of-Products (SOP)
  - Product-of-Sum (POS)
- **Circuit Simplification**
  - Karnaugh Map
  - Half Adder and Full Adder
- **Designing Large Logic Circuits**
  - Shannon's Expansion Theorem
  - Topological ordering and Hardware Description Program
- **Logic Games**
  - Logic Diagrams and Puzzles

Background Reading: Chapters 4 and 5 in Textbook

One-bit Adder



Truth Table

A	B	C	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

SOP Forms:

$$S = \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC$$

$$C_{out} = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Karnaugh Map

- Also known as K-map
- Invented in 1953 by American Electrical
- Engineer Maurice Karnaugh



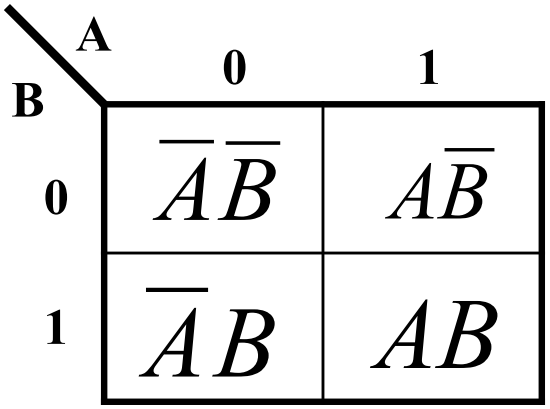
- K-maps provide a graphical approach to reduce Boolean equation without Boolean algebra (thereby simplifying logic circuits).
- Instead of Boolean algebra analysis, you transfer logic values from a Boolean statement or a truth table into a K-map.
- The arrangement of 0's and 1's within the map helps you to visualize the logic relationships between the variables and also leads to a systematic approach to a *simplified* Boolean statement in SOP form.

Karnaugh Map

- **Karnaugh maps, or K-maps, are often used to simplify logic problems with 2, 3 or 4 variables.**

Cell = 2^n ,where n is a number of variables

- **For the case of 2 variables, we form a map consisting of 2^2=4 cells**
- **as shown in Figure**



Karnaugh Map

- **3 variables Karnaugh map**

Cell = 2^3=8

C \ AB	00	01	11	10
0	0 $\overline{A}\,\overline{B}\,\overline{C}$	2 $\overline{A}B\overline{C}$	6 $AB\overline{C}$	4 $A\overline{B}\,\overline{C}$
1	1 $\overline{A}\,\overline{B}C$	3 $\overline{A}BC$	7 ABC	5 $A\overline{B}C$

The numbers in red denote the row number in the truth table from top to bottom.

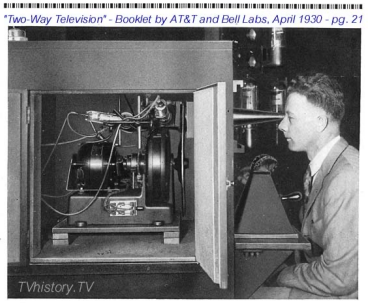
Karnaugh Map

- **4 variables Karnaugh map**

CD \ AB	00	01	11	10
00	0	4	12	8
01	1	5	13	9
11	3	7	15	11
10	2	6	14	10

The numbers in red denote the row number in the truth table from top to bottom.

Karnaugh Map (Logic Game)



DETAILS of the color-television receiver.

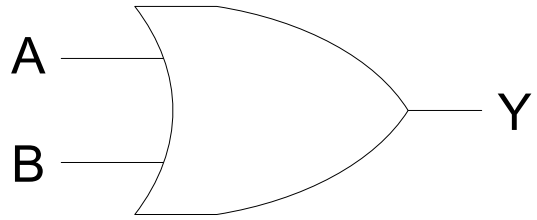
- **Setting up the Stage:**
  - Complete the Karnaugh map (K-map) by entering a '1' (or a '0') in each of the cells appropriately.
  - Verify that the labeling uses **Gray code**

- **Playing the K-map Game:**
  - In the K-map, find groups containing adjacent cells of 1's with each group size as large as possible
  - The number of 1's in a group is twos, fours, or eights etc.
  - Find as few groups as possible in the K-map

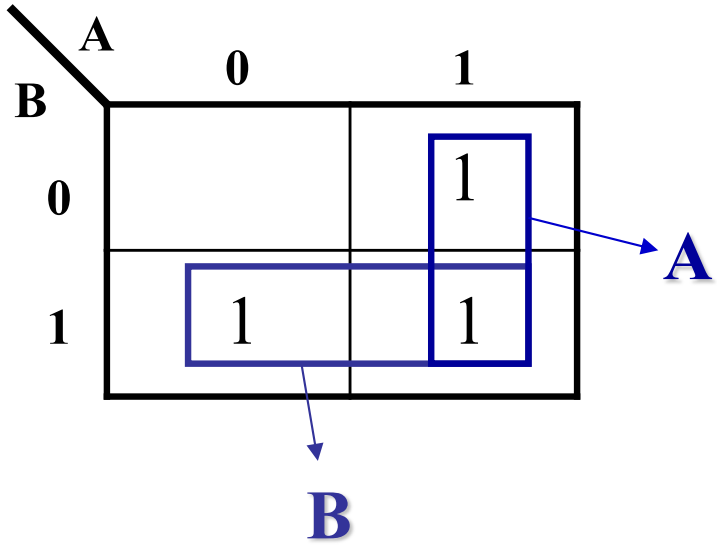
Gray Code (by bit width)	
2-bit	4-bit
00	0000
01	0001
11	0011
10	0010
	0110
	0111
3-bit	0101
000	0100
001	1100
011	1101
010	1111
110	1110
111	1010
101	1011
100	1001
	1000

Example of 2-variable K-map

- 2-variable Karnaugh maps are trivial but can be used to introduce
- the methods you need to learn. The map for a 2-input OR gate
- looks like this:



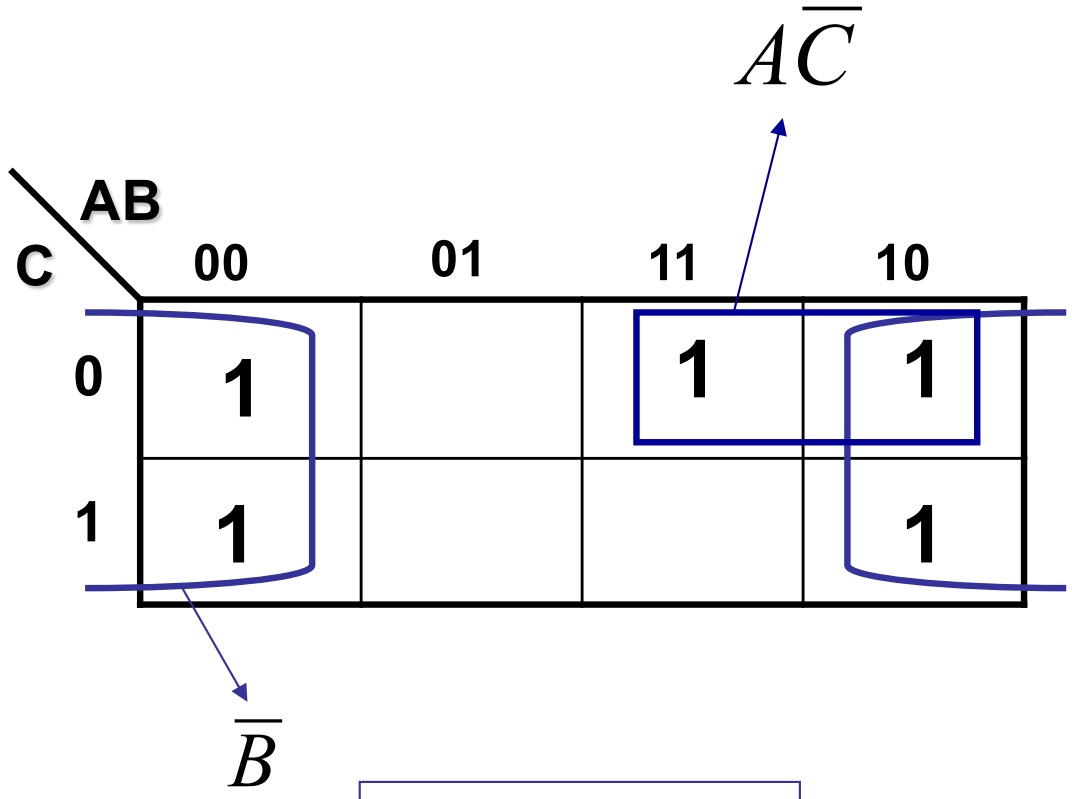
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



A+B

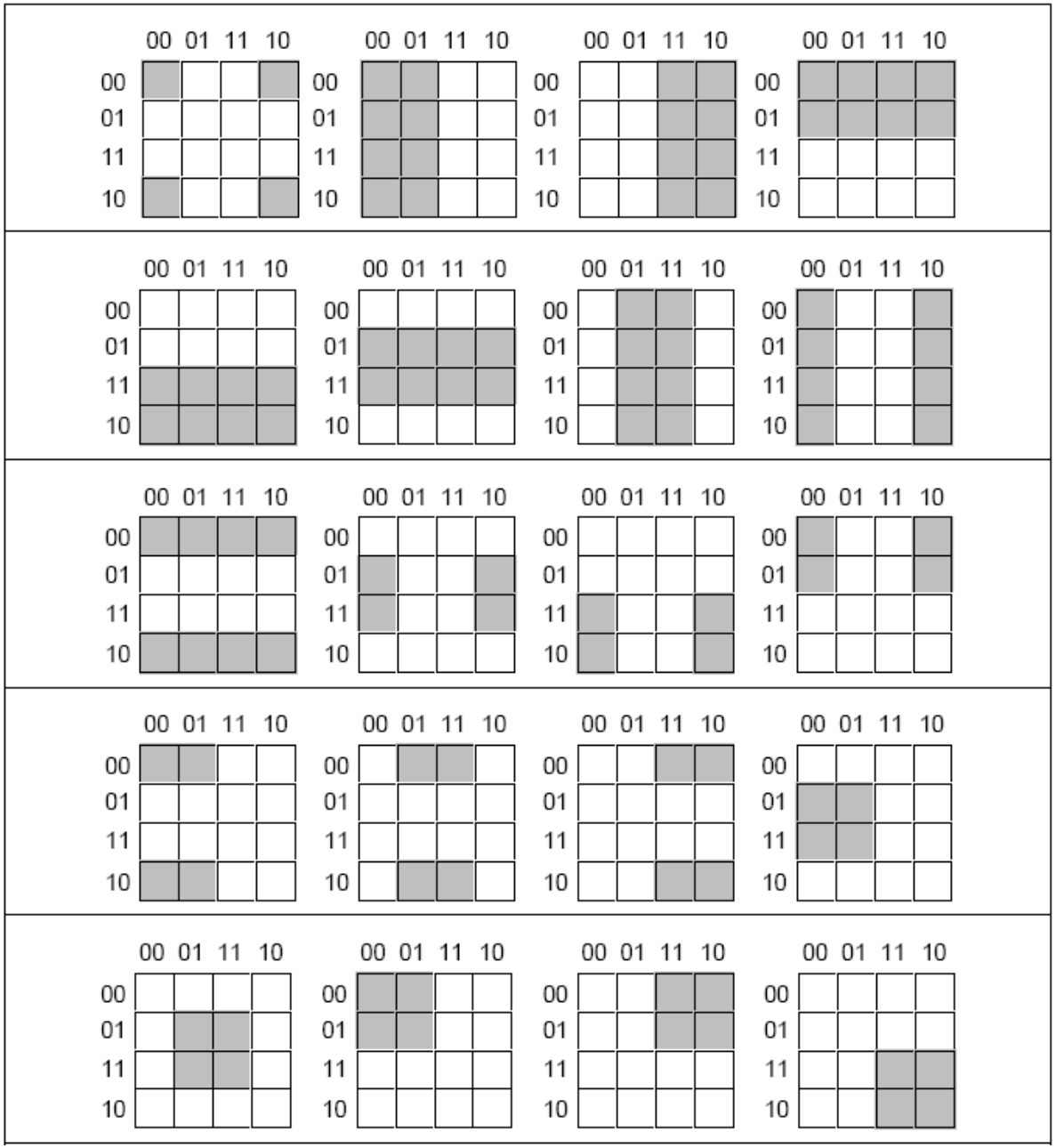
Example of 3-variable K-map

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

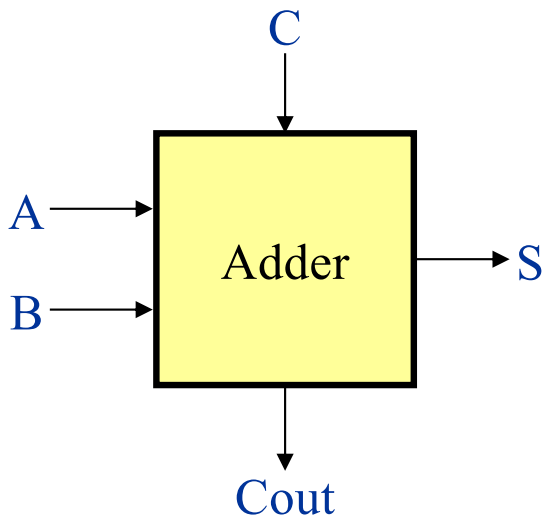


$$\overline{B} + A\overline{C}$$

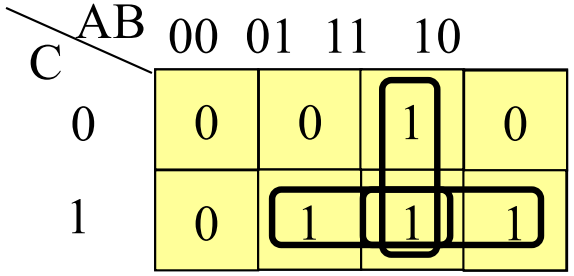
Example of 4-Variable K-map's Allowed Patterns



Simplify the One-bit Full Adder by K-Map



A	B	C	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Karnaugh Map for Cout

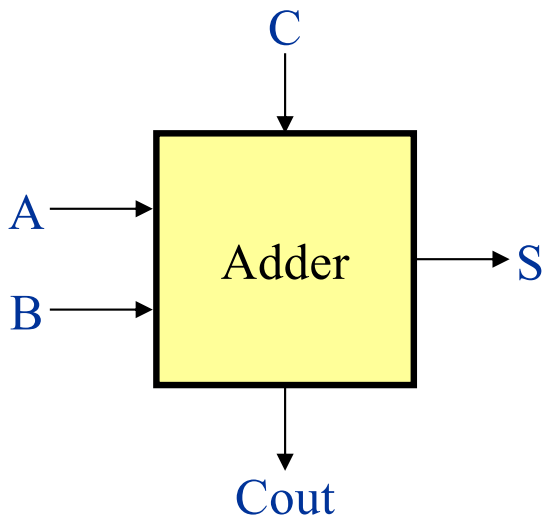
Group all the 1's in the K-Map with as few groups as possible and each group as big as possible.

Cout = AB + BC + AC

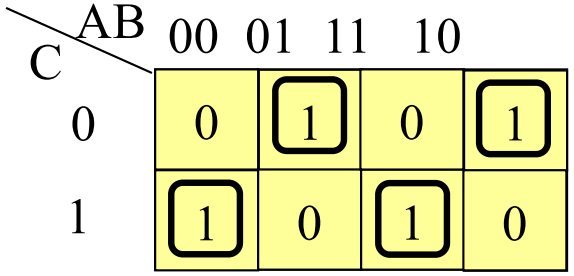
The original one:

$$C_{out} = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Simplify the One-bit Full Adder by K-Map



A	B	C	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

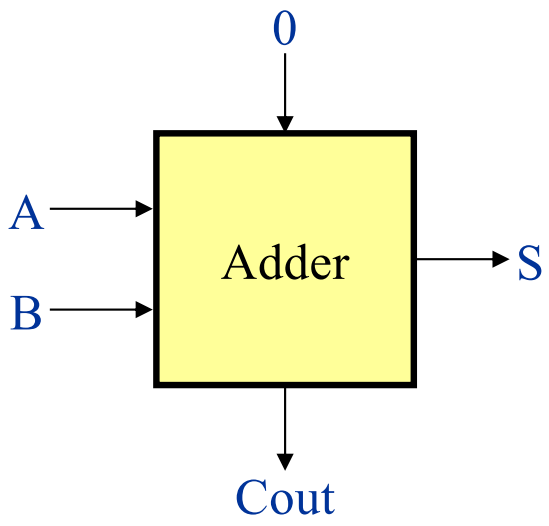


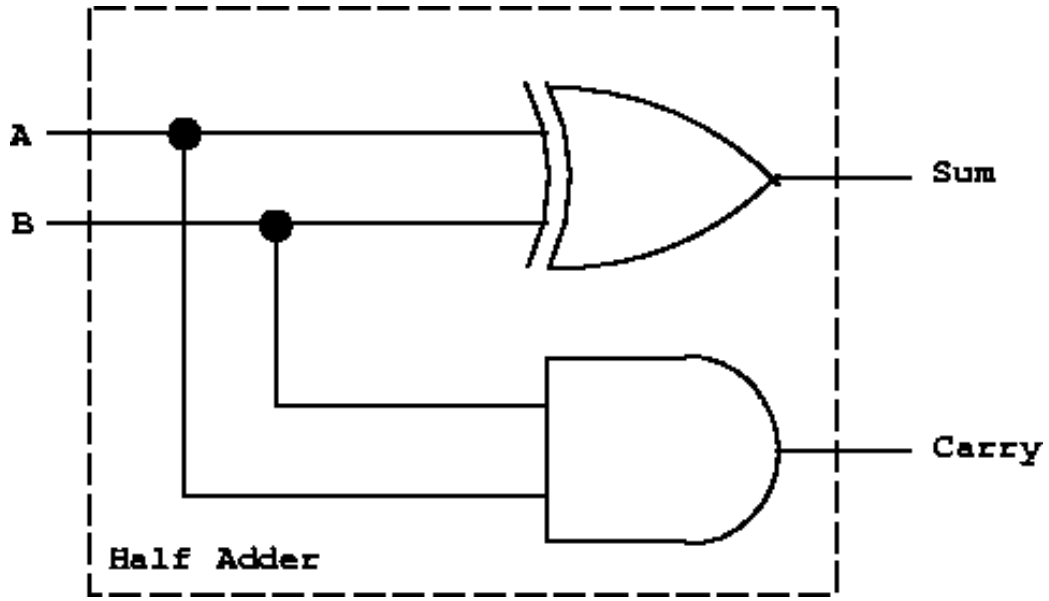
Karnaugh Map for Cout

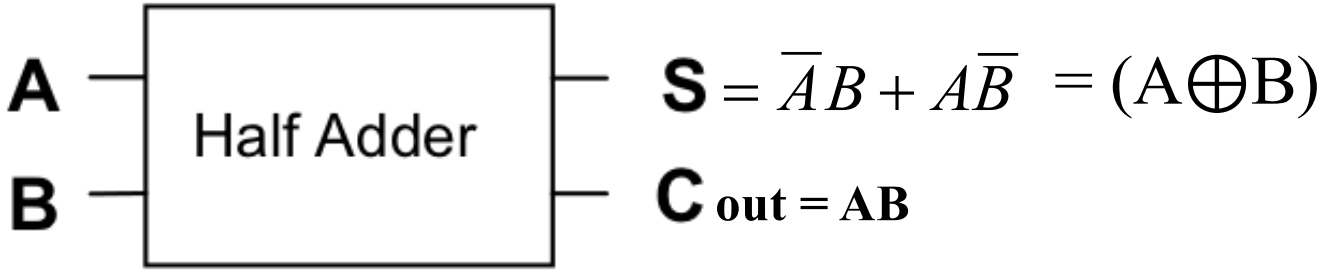
Group all the 1's in the K-Map with as few groups as possible and each group as big as possible.

$$S = \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC$$

Special Case: Half Adder (C=0)

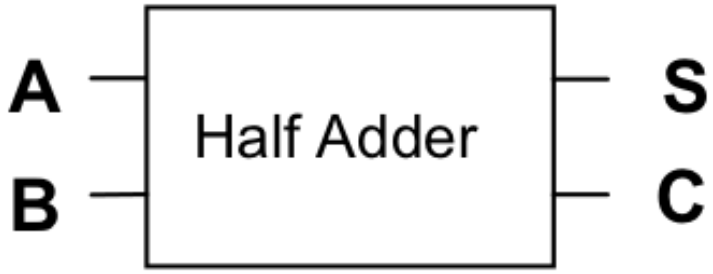






The *S* logical function simplifies to a logical XOR function and the C_{out} function simplifies to a logical AND function.

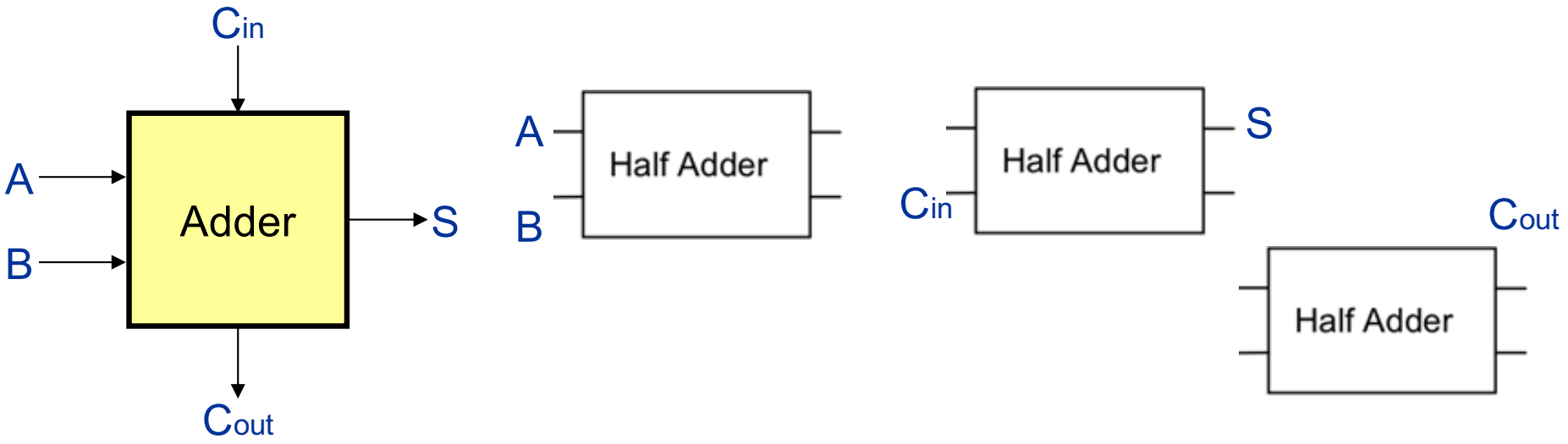
Half Adder as Universal Gate



The *S* logical function simplifies to a logical XOR function and the C_{out} function simplifies to a logical AND function.

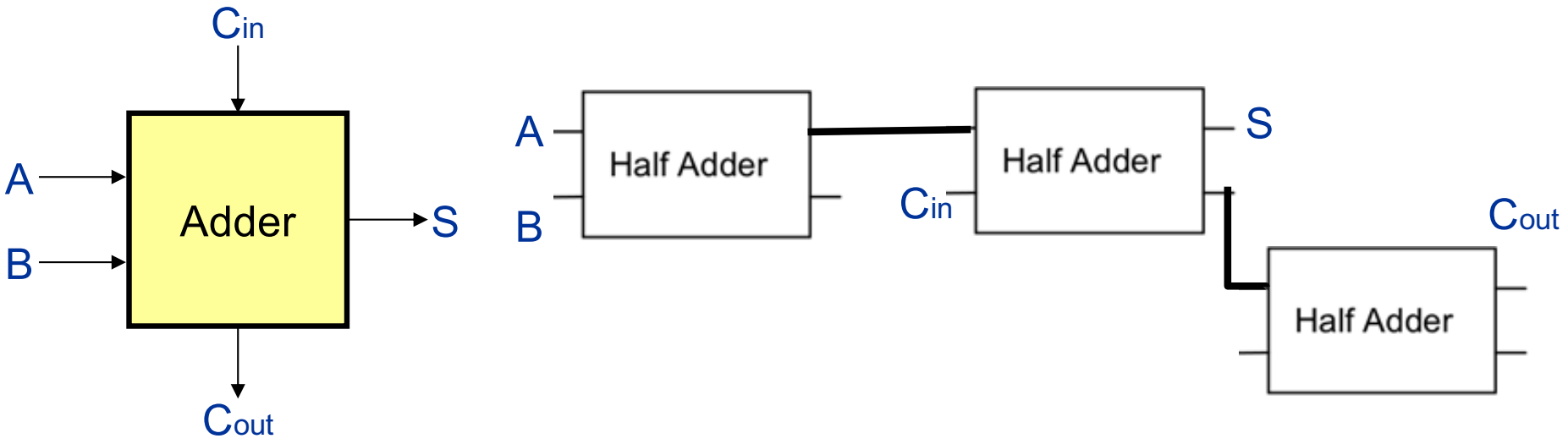
Synthesize One-bit Full Adder with Half Adders

Inter-connect Half Adders to produce the desired output of a Full Adder



Synthesize One-bit Full Adder with Half Adders

Inter-connect Half Adders to produce the desired output of a Full Adder

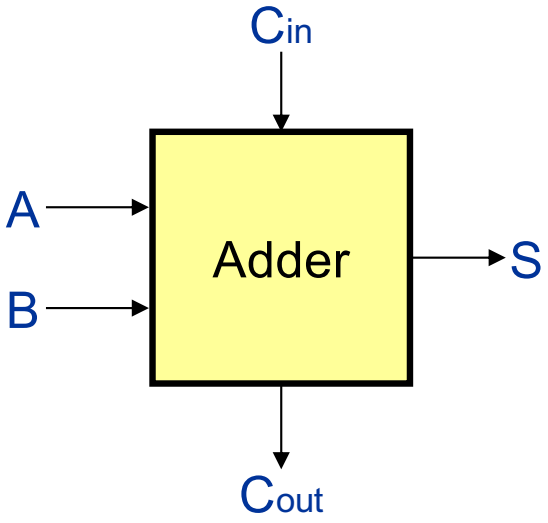


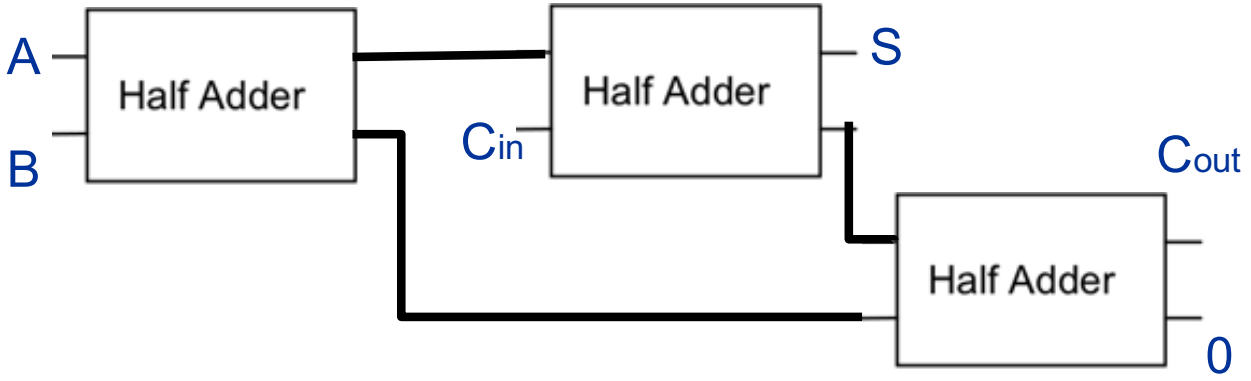
$$S = \overline{A}\,\overline{B}\,C_{in} + \overline{A}\,B\,\overline{C}_{in} + A\,\overline{B}\,\overline{C}_{in} + ABC_{in}$$

$$= \mathrm{A} \oplus \mathrm{B} \oplus \mathrm{Cin}$$

Synthesize One-bit Full Adder with Half Adders

Inter-connect Half Adders to produce the desired output of a Full Adder



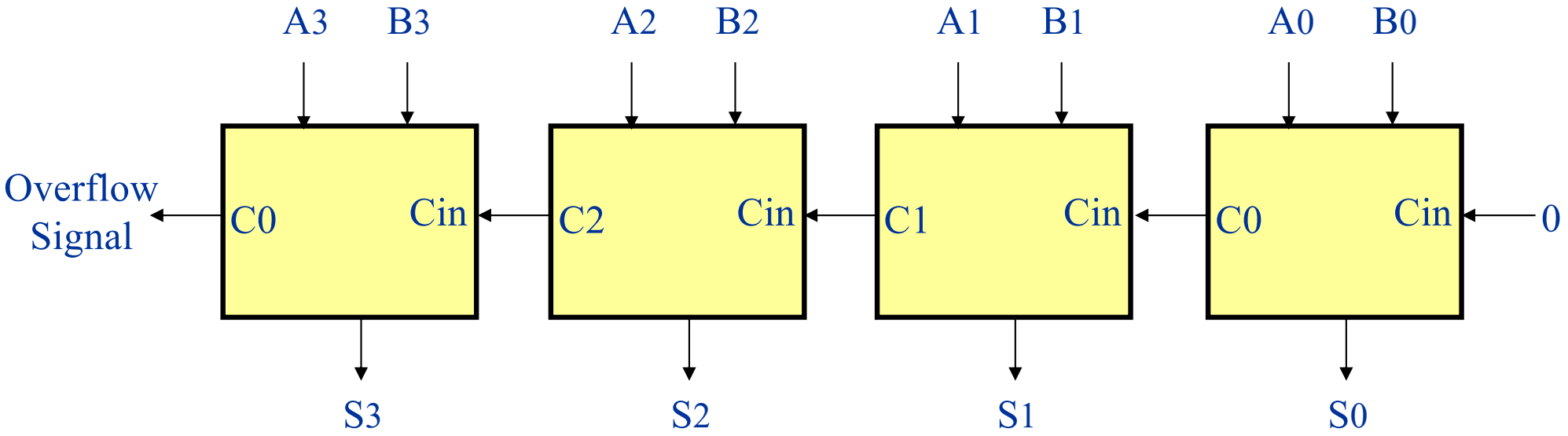


Constant used to detect fault

$$S = \overline{A}\,\overline{B}C_{in} + \overline{A}B\overline{C}_{in} + A\overline{B}\,\overline{C}_{in} + ABC_{in}$$
$$= A \oplus B \oplus C_{in}$$

$$\begin{aligned} C_{out} &= AB + BC_{in} + AC_{in} \\ &= AB + (A \oplus B)\, C_{in} \\ &= (A \oplus B)\, C_{in} \oplus AB \oplus (A \oplus B)\, ABC_{in} \\ &= (A \oplus B)\, C_{in} \oplus AB \oplus (AB \oplus AB)\, C_{in} \\ &= (A \oplus B)\, C_{in} \oplus AB \oplus (0)\, C_{in} \\ &= (A \oplus B)\, C_{in} \oplus AB \oplus 0 \\ &= (A \oplus B)\, C_{in} \oplus AB \\ &= AC_{in} \oplus BC_{in} \oplus AB \end{aligned}$$

Four-bit Adder

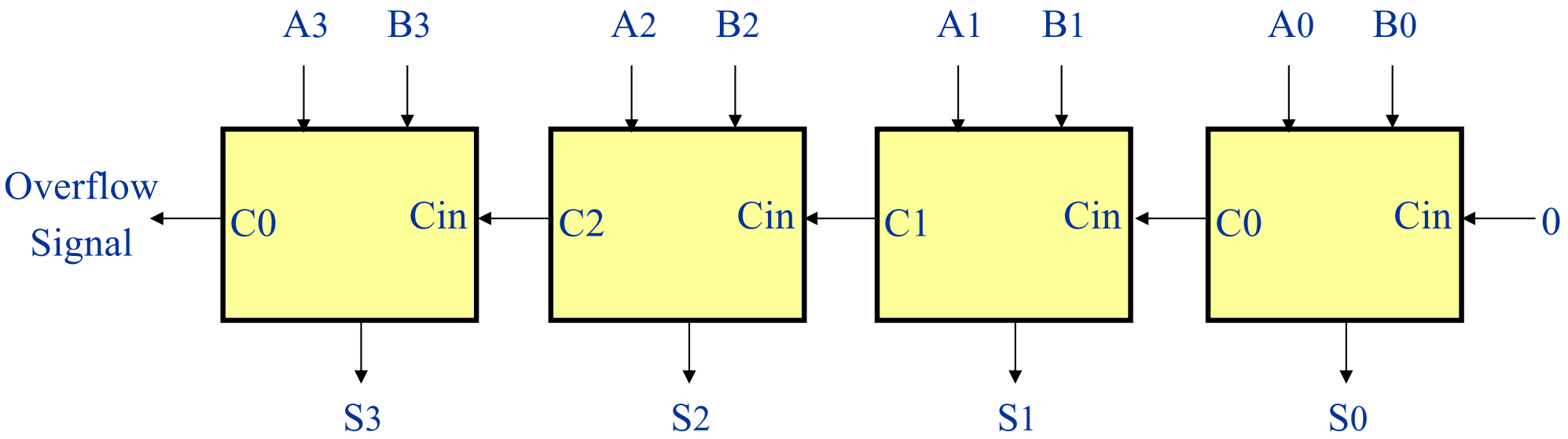


Adding two 4-bit numbers by an interconnection of four one-bit Full Adders

Also called the Ripple Carry Adder as the Carry-out *ripples* from right to left.

A zero is input on the righthand-side and a zero on the lefthand-side denotes normal operation

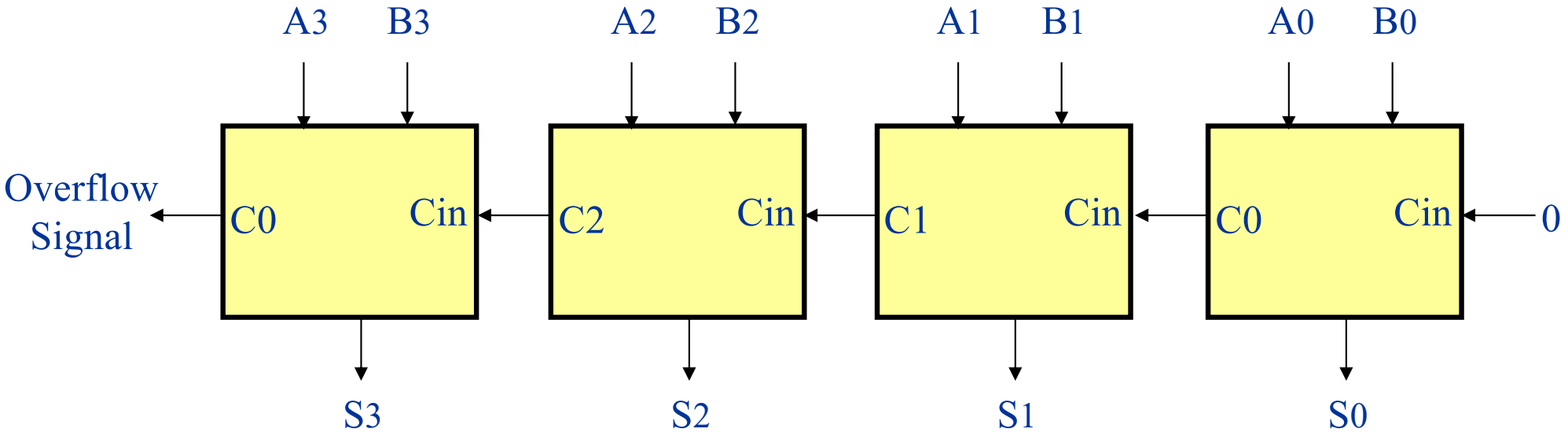
Probability of Overflow in Four-bit Adder



Supposed the inputs A and B have bits that are uniformly generated.

What is the probability that the “overflow signal” gives “True” as output?

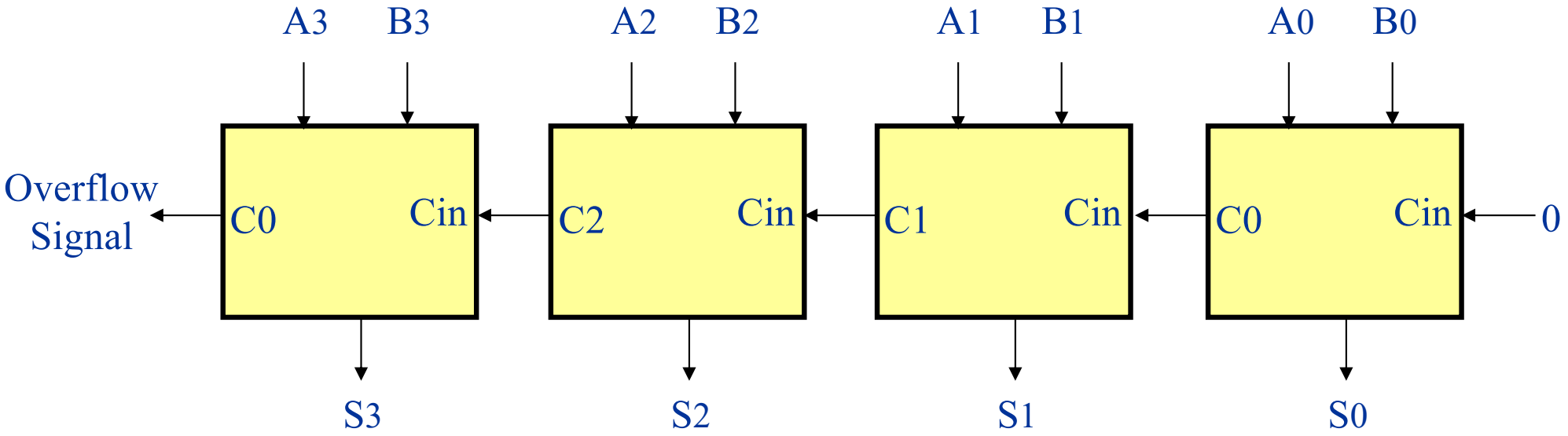
Probability of Overflow in Four-bit Adder



Supposed the inputs A and B have bits that are uniformly generated.
What is the probability that the “overflow signal” gives “True” as output?

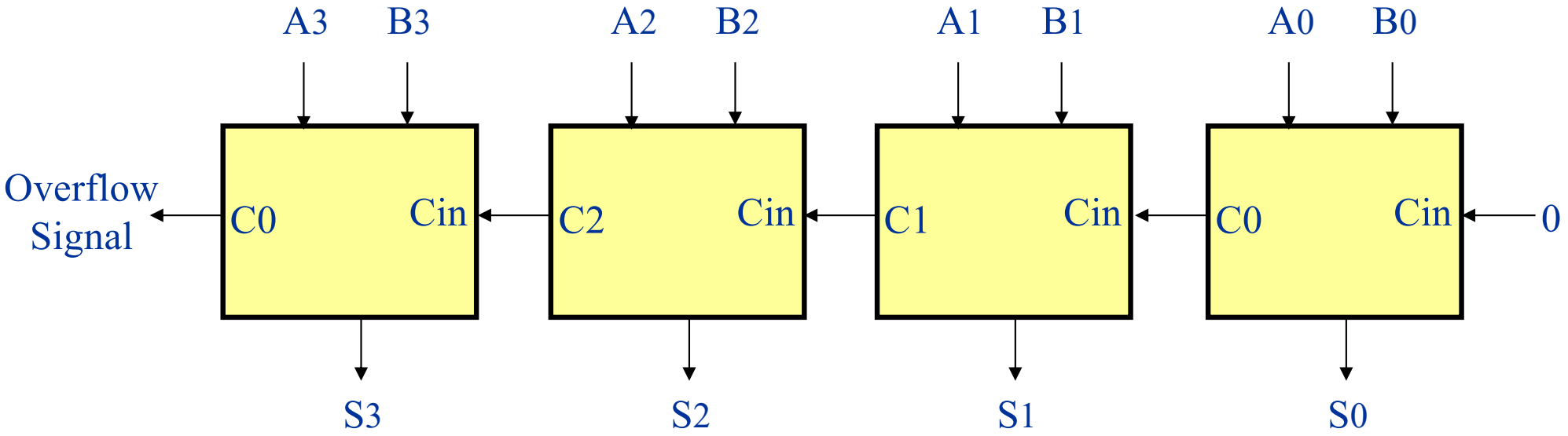
Look at addition table, deduce as (N choose floor(N/2)) / N^2
Note that Sum gives the permutation group and the mod (log(A)) finite field
The Overflow Signal draws up the Pascal Triangle

Synthesis a 12-hour Clock from Adders



The “overflow signal” tells AM or PM

Fault-tolerant Four-bit Adder



How to overcome 1-bit error?

Detecting error occurence as Stuck-at Fault or zero-one flip fault

- http://www.nytimes.com/1994/11/24/business/company-news-flaw-undermines-accuracy-of-pentium-chips.html

Majority Voting

$$C_{out} = AB + BC_{in} + AC_{in}$$
$$= C_{in}A \oplus BC_{in} \oplus AB$$

Trouble-shooting the full adder and the 4-bit adder with RM code

What are co-factors of majority organ?

Differentiate the polynomial

Reed Muller Polynomial is a path through the Hasse diagram

Logic puzzle to solve three variables given two equations:

What is A, B and C given the system of equations (one is quadratic):

$AB \oplus AC \oplus BC = 1$

$A \oplus B \oplus C = 1$

Majority Voting and Full Adder

Lemma 4.1 *If a, b and c are three binary inputs, then* $M(a, b, M(a, b, c)) = M(a, b, c)$

Proof: $M(a, b, M(a, b, c)) = ab + b(ab + bc + ac) + a(ab + bc + ac)$
$= ab + bc + ac + abc$
$= M(a, b, c)$ Q.E.D

As a consequence of Lemma 4.1, we have $M(a, M(a, b, c), c) = M(a, b, c)$ and $M(M(a, b, c), b, c) = M(a, b, c)$.

Lemma 4.2 *Let a,b and c be three binary inputs. Then* $M(a, b, \overline{M(a, b, c)}) = M(a, b, \bar{c})$

Proof: $M(a, b, \overline{M(a, b, c)}) = ab + a(\bar{a}\bar{b} + \bar{b}\bar{c} + \bar{a}\bar{c}) + b(\bar{a}\bar{b} + \bar{b}\bar{c} + \bar{a}\bar{c})$
$= ab + a\bar{b}\bar{c} + \bar{a}b\bar{c}$
$= a(b + \bar{b}\bar{c}) + b(a + \bar{a}\bar{c})$
$= a(b + \bar{c}) + b(a + \bar{c})$
$= M(a, b, \bar{c})$ Q.E.D

As a consequence of Lemma 4.2, we have $M(a, \overline{M(a, b, c)}, c) = M(a, \bar{b}, c)$ and $M(\overline{M(a, b, c)}, b, c) = M(\bar{a}, b, c)$

Lemma 4.3 *Let f_1, f_2 and f_3 be three Boolean functions such that f_1 and f_2 satisfy $f_1 f_2 = f_1$ and $f_1 + f_2 = f_2$. Then* $M(f_1, f_2, f_3) = f_1 + f_2 f_3$

Proof: $M(f_1, f_2, f_3) = f_1 f_2 + f_1 f_3 + f_2 f_3$
$= f_1 + (f_1 + f_2) f_3$
$= f_1 + f_2 f_3$ Q.E.D

4.1.2 Application of the Majority Logic Results to a Ripple Carry Adder

An interesting consequence of Lemma 4.2 is Lemma 4.4 described next. This lemma establishes that carry generation requires one majority gate and sum generation requires just two majority gates plus one inverter in a one-bit full adder. Let a_i, b_i and c_i be inputs to a full adder and let s_i and c_{i+1} be its outputs.

Lemma 4.4 *A 1-bit full adder can be realized using 3 majority gates and 1 inverter.*

Proof From the results in [4], we have

$$c_{i+1} = M(a_i, b_i, c_i) \tag{4.1}$$

The sum s_i can be expressed in three equivalent formats as given by Eqs. (4.2)–(4.4).

$$s_i = M\left(\overline{M(a_i, b_i, c_i)}, M\left(a_i, b_i, \overline{c_i}\right), c_i\right) \tag{4.2}$$

If b_i in Eq. (4.2) is complemented instead of c_i, we have

$$s_i = M\left(\overline{M(a_i, b_i, c_i)}, M(a_i, \overline{b_i}, c_i), b_i\right) \tag{4.3}$$

Similarly,

$$s_i = M\left(\overline{M(a_i, b_i, c_i)}, M(\overline{a_i}, b_i, c_i), a_i\right) \tag{4.4}$$

Using Lemma 4.2 in Eq. (4.4), we have

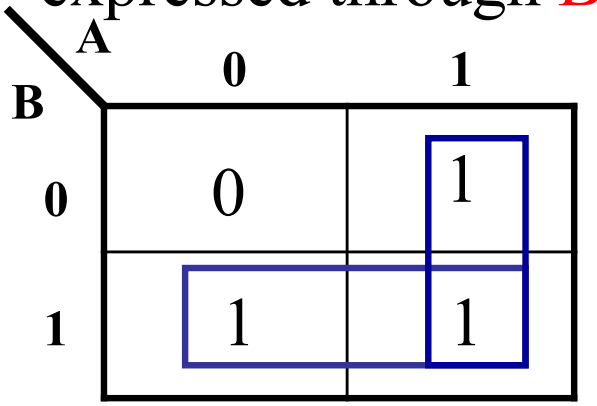
$$s_i = M\left(\overline{M(a_i, b_i, c_i)}, M\left(\overline{M(a_i, b_i, c_i)}, b_i, c_i\right), a_i\right) \tag{4.5}$$

Q.E.D

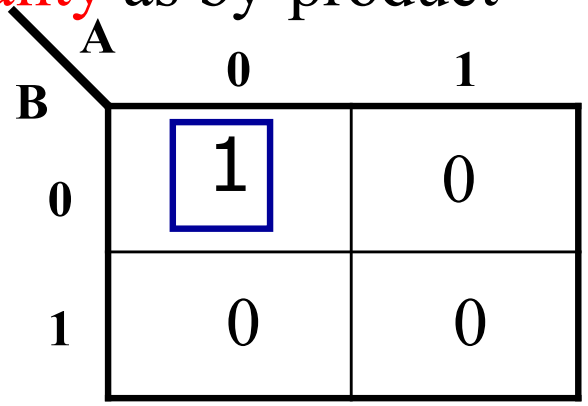
A one-bit full adder that incorporates appropriate clocking is shown in Fig. 4.2. The D-latch convention presented in [5] enables us to obtain the total circuit delay. One D-latch (namely, $D0$) is used to indicate that one-quarter of a clock is required to apply the inputs to the majority logic. One-fourth clock zone delay is assumed when a majority gate is immediately followed by an inverter or vice-versa ($D1$ is introduced at the output of inverter that follows the majority gate [5]). Proceeding this way, we have a total circuit delay of 1 clock (4 clock zones) for generating S_i as well as C_{i+1} for a 1-bit adder.

Inverse of Boolean Function by Karnaugh Map

- K-map gives the complementary of the Boolean function expressed through Boolean duality as by-product



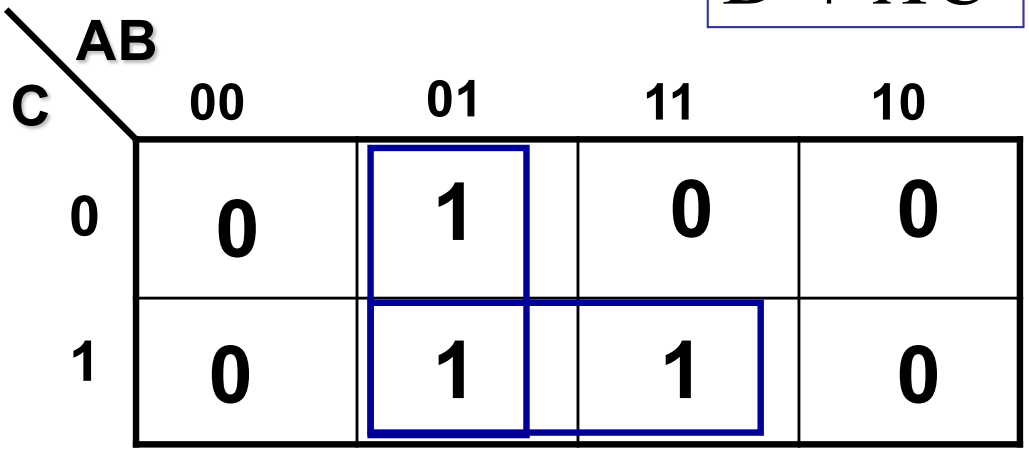
A+B



$\overline{A}\,\overline{B}$

Recall how we have used the De Morgan's Theorem earlier

- What is the complementary of $\overline{B} + A\overline{C}$?



- Reduction to smallest possible Product-of-Sum (POS) form.

Fourier Transform (Reed Muller Canonical Form)

Shannon expansion

$f = (x\ f_x) \oplus (x'\ f_{x'})$

$= (x\ f_x) \oplus (1 \oplus x)\ f_{x'}$

$= (x\ f_x) \oplus f_{x'} \oplus (x\ f_{x'}) = f_{x'} \oplus x(\ f_{x'} \oplus f_x\ )$

Combining Shannon expansion and finite field GF(2) using half-adder leads to:

- new GF(2) polynomial algebra (Fourier transform)
- ability to test circuit for faults (Reed Muller testing problem), e.g., test faults in 4-bit Ripple Carry Adder
- Tao's majority voting results

Test code – Reed-Muller code

Test faults in Ripple Carry Adder

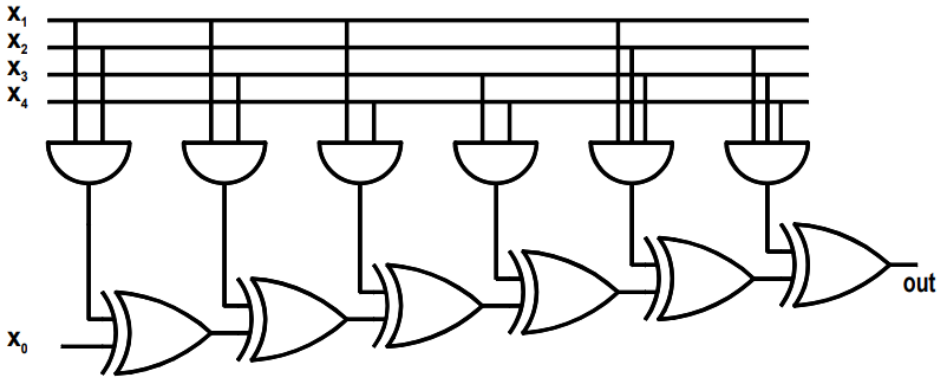


Figure 19.2: Circuit implementation of a Reed-Muller AND/XOR function.

Outline

- **Basics of Combinational Circuits**
  - Boolean Algebra
  - Truth Table & Logic Gates
- **Canonical Implementation**
  - Sum-of-Products (SOP)
  - Product-of-Sum (POS)
- **Circuit Simplification**
  - Karnaugh Map
  - Half Adder and Full Adder
- **Designing Large Logic Circuits**
  - Shannon's Expansion Theorem
  - Topological ordering and Hardware Description Program
- **Logic Games**
  - Logic Diagrams and Puzzles

Background Reading: Chapters 4 and 5 in Textbook

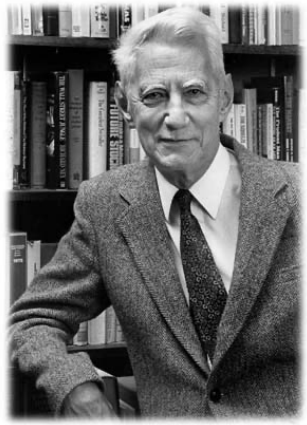
Shannon's Expansion Theorem

- Background
  - You've done Boolean algebra, hand manipulations, Karnaugh maps to simplify…
  - But this is **not sufficient** for real designs!

- Example: a multiplier of two 16-bit numbers
  - It has 32 inputs.
  - Its Karnaugh map has $2^{32} = 4{,}294{,}967{,}296$ squares
    - This is too big!
  - There must be a better way…

Not only do we have combinatorial barrier, the game of K-map belongs to the classical **Set Cover Problem** (NP-complete, 1972) which is a computational barrier!

Shannon's Expansion Theorem



- In calculus, you can represent complex functions like e^x using simpler functions.
  - If you can only use $1, x, x^2, x^3, \ldots$ as the pieces …
  - … turns out $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$
- It corresponds to the **Taylor series expansion**.
  - $f(x) = f(0) + \frac{f'(0)}{1!} + \frac{f''(0)}{2!} + \frac{f'''(0)}{3!} + \cdots$

Question: Anything like this for Boolean functions?

Yes. It is called **Shannon Expansion**.

Shannon's Expansion Theorem

- Proposed by **Claude Shannon**, the father of information theory.
- Suppose we have a function $F(x_1, x_2, \ldots, x_n)$.
- Define **a new function** if we set one of the $x_i = const$
  - $F(x_1, x_2, \ldots, x_i = 1, \ldots, x_n)$
  - $F(x_1, x_2, \ldots, x_i = 0, \ldots, x_n)$
- Example: $F(x, y, z) = xy + x\bar{z} + y(\bar{x}z + \bar{z})$
  - $F(x = 1, y, z) = y + \bar{z} + y\bar{z}$
  - $F(x, y = 0, z) = x\bar{z}$

Note: this is a new function, that no longer depends on the variable x_i.

Shannon's Expansion Theorem

- Turns out to be an incredibly useful idea.
- It is also known as **Shannon cofactor** with respect to x_i.
  - We write $F(x_1, x_2, \ldots, x_i = 1, \ldots, x_n)$ as F_{x_i}.
    - We call it **positive cofactor**.
  - We write $F(x_1, x_2, \ldots, x_i = 0, \ldots, x_n)$ as $F_{\overline{x_i}}$.
    - We call it **negative cofactor**.
  - Often, just write them as $F(x_i = 1)$ and $F(x_i = 0)$.

- Why are these useful functions to get from F?

Shannon's Expansion Theorem

- Why we care: **Shannon Expansion Theorem**
- Given any Boolean function $F(x_1, x_2, \ldots, x_n)$ and pick any x_i in F's inputs, F can be represented as
  $F(x_1, x_2, \ldots, x_n) = x_i \cdot F(x_i = 1) + \overline{x_i} \cdot F(x_i = 0)$

Proof:

Consider any $(x_1, x_2, \ldots, x_n) \in \{0,1\}^n$

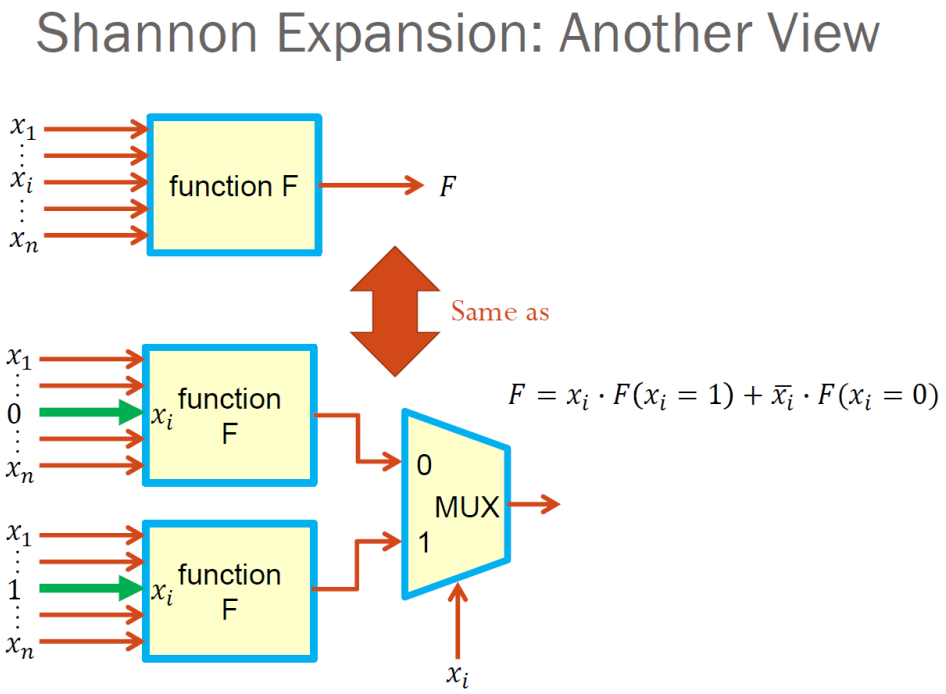
- If $x_i = 1$

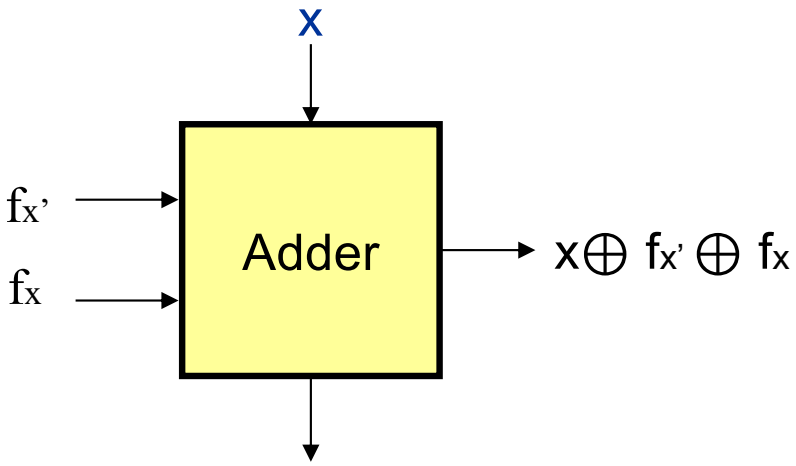
  $F(\ldots, x_i = 1, \ldots) = F(x_i = 1) = 1 \cdot F(x_i = 1) + \overline{1} \cdot F(x_i = 0)$

- If $x_i = 0$

  $F(\ldots, x_i = 0, \ldots) = F(x_i = 0) = 0 \cdot F(x_i = 1) + \overline{0} \cdot F(x_i = 0)$

Shannon's Expansion Theorem





fx fx' ⊕ x(fx' ⊕ fx)

fx fx' is the consensus of f with respect to x

Shannon's Expansion Theorem

- Can do it on **more than one** variable, too.
  - Just keep on applying the theorem on each variable.

- Example: Expand $F(x, y, z, w)$ around x and y
  - First, expand around x:

$$F(x, y, z, w) = x \cdot F(x = 1) + \bar{x} \cdot F(x = 0)$$

  - Then, expand cofactors $F(x = 1)$ and $F(x = 0)$ around y:

$$F(x = 1) = y \cdot F(x = 1, y = 1) + \bar{y} \cdot F(x = 1, y = 0)$$
$$F(x = 0) = y \cdot F(x = 0, y = 1) + \bar{y} \cdot F(x = 0, y = 0)$$

  - Final result:

$$F(x, y, z, w)$$
$$= xy \cdot F(x = 1, y = 1) + x\bar{y} \cdot F(x = 1, y = 0)$$
$$+\bar{x}y \cdot F(x = 0, y = 1) + \bar{x}\bar{y} \cdot F(x = 0, y = 0)$$

Shannon's Expansion Theorem

- There is notation for these multiple-variable expansions as well.
- Shannon cofactor with respect to x_i and x_j:
  - Write $F(x_1, \ldots, x_i = 1, \ldots, x_j = 0, \ldots, x_n)$ as $F_{x_i \overline{x_j}}$.
  - The same for any number of variables $x_i, x_j, x_k, \ldots$
  - Notice that order does **not** matter: $(F_x)_y = (F_y)_x = F_{xy}$.

- For the previous example:

$$F(x, y, z, w) = xy \cdot F_{xy} + x\bar{y} \cdot F_{x\bar{y}} + \bar{x}y \cdot F_{\bar{x}y} + \bar{x}\bar{y} \cdot F_{\bar{x}\bar{y}}$$

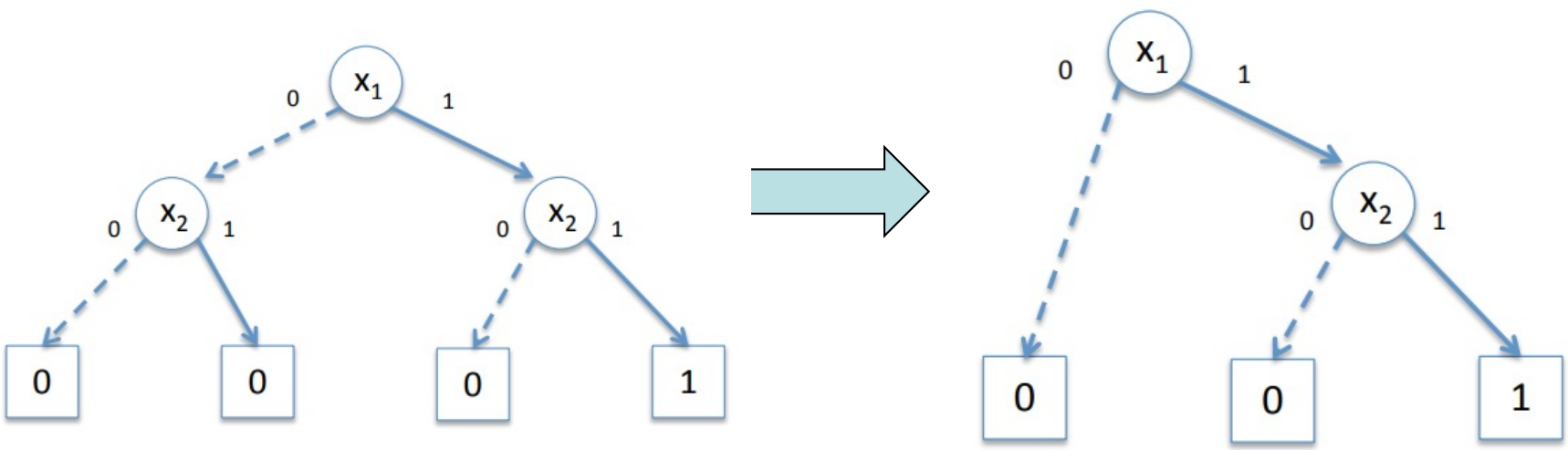
- Again, remember: each of the cofactors is a **function**, not a number.
  - $F_{xy} = F(x = 1, y = 1, z, w)$ is a Boolean **function** of only z and w.

Binary Decision Diagram

- Binary decision diagram (BDD) is a data structure representation of a Boolean function in the form of a graph, directly derivable from Shannon's expansion theorem.
- **BDD of an n-variable Boolean function is a tree:**
  - Root node is any input variable.
  - All nodes in a level are labeled by the same input variable.
  - Each node has two outgoing edges, labeled as 0 and 1 indicating the state of the node variable.
  - Leaf nodes carry fixed 0 and 1 labels.
  - Levels from root to leaf nodes represent an ordering of input variables.
  - If we trace a path from the root to any leaf, the label of the leaf gives the value of the Boolean function when inputs are assigned the values from the path.
- **References:**

  - C. Y. Lee, "Representation of Switching Circuits by Binary Decision Diagrams," *Bell Syst. Tech J.*, vol. 38, pp. 985-999, July 1959.
  - S. Akers, "Binary Decision Diagrams," *IEEE Trans. Computers*, vol. C-27, no. 6, pp. 509-516, June 1978

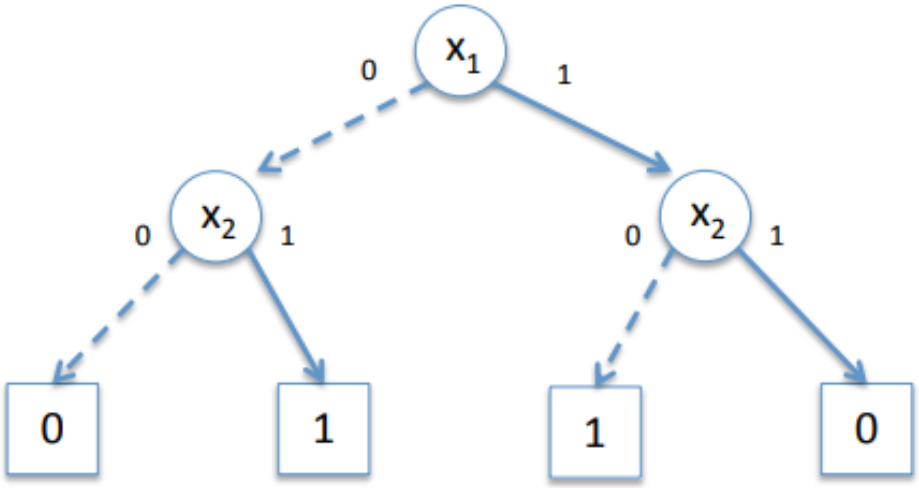
From Decision Tree to Binary Decision Diagram

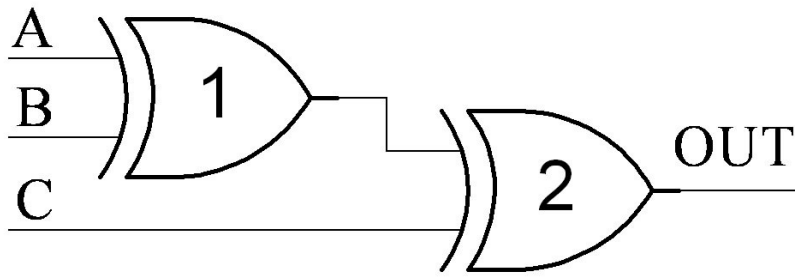


Allow sharing of identical subtrees in the decision tree to condense the decision tree into a binary decision diagram (also a rooted tree) that is compact (i.e., save memory space by compressing the Truth Table)

Example of deriving the Binary Decision Diagram of AND logical function from its decision tree

From Decision Tree to Binary Decision Diagram



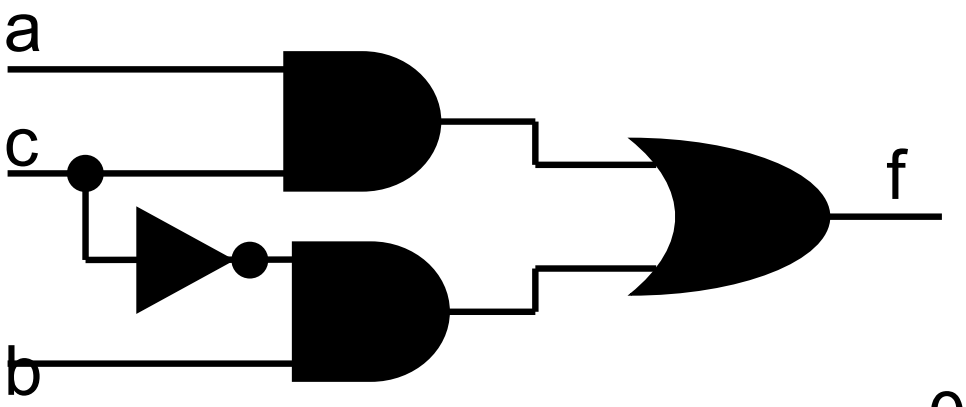


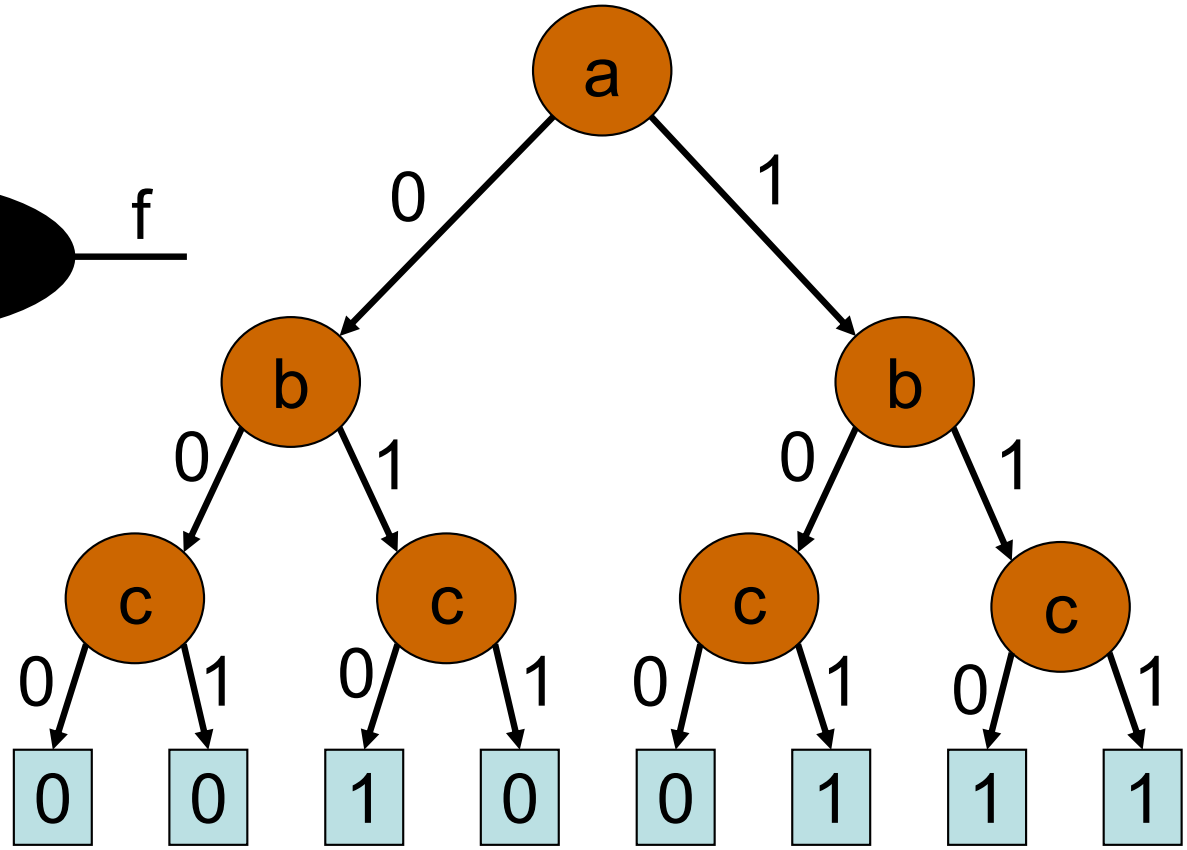
Example of the XOR function where the Binary Decision Diagram is the decision tree

What is the insight you can deduce?

Draw the Binary Decision Diagram of the parity check logic circuit (in fact, simply a multi-XOR application)

Binary Decision Tree

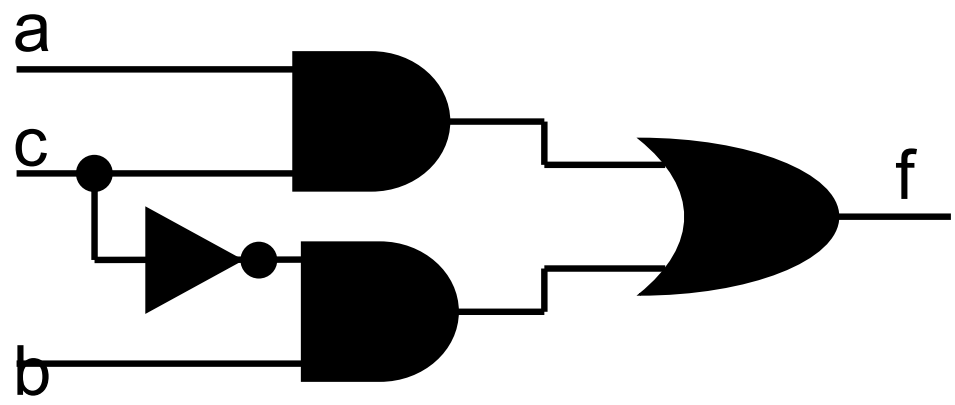


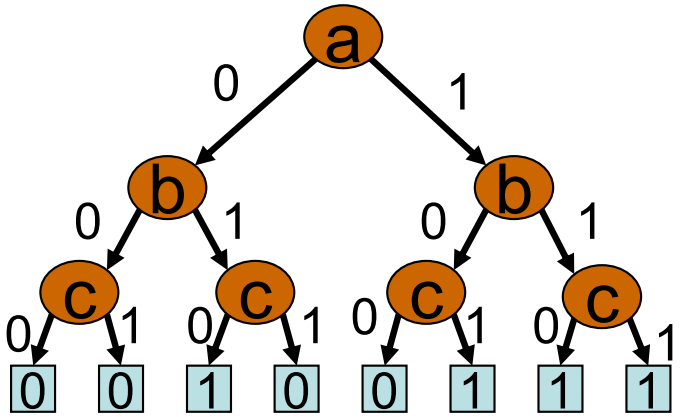


Graph representation of a Boolean function or equivalently the Truth Table

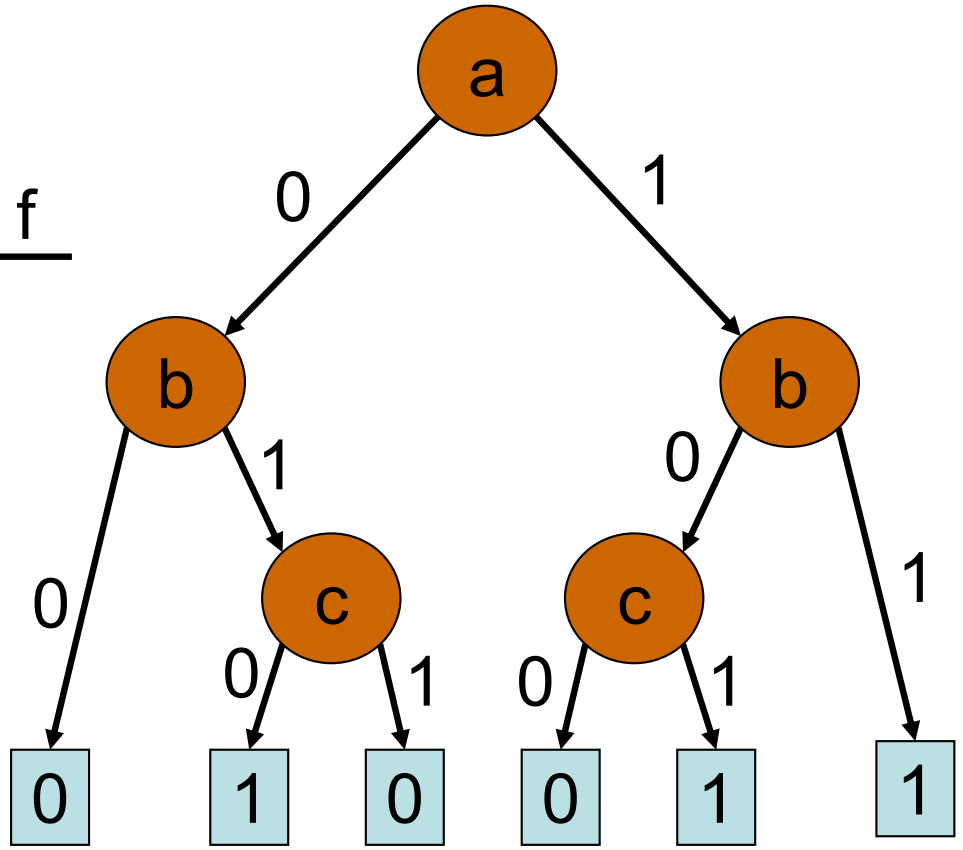
Leaf nodes

Binary Decision Diagram



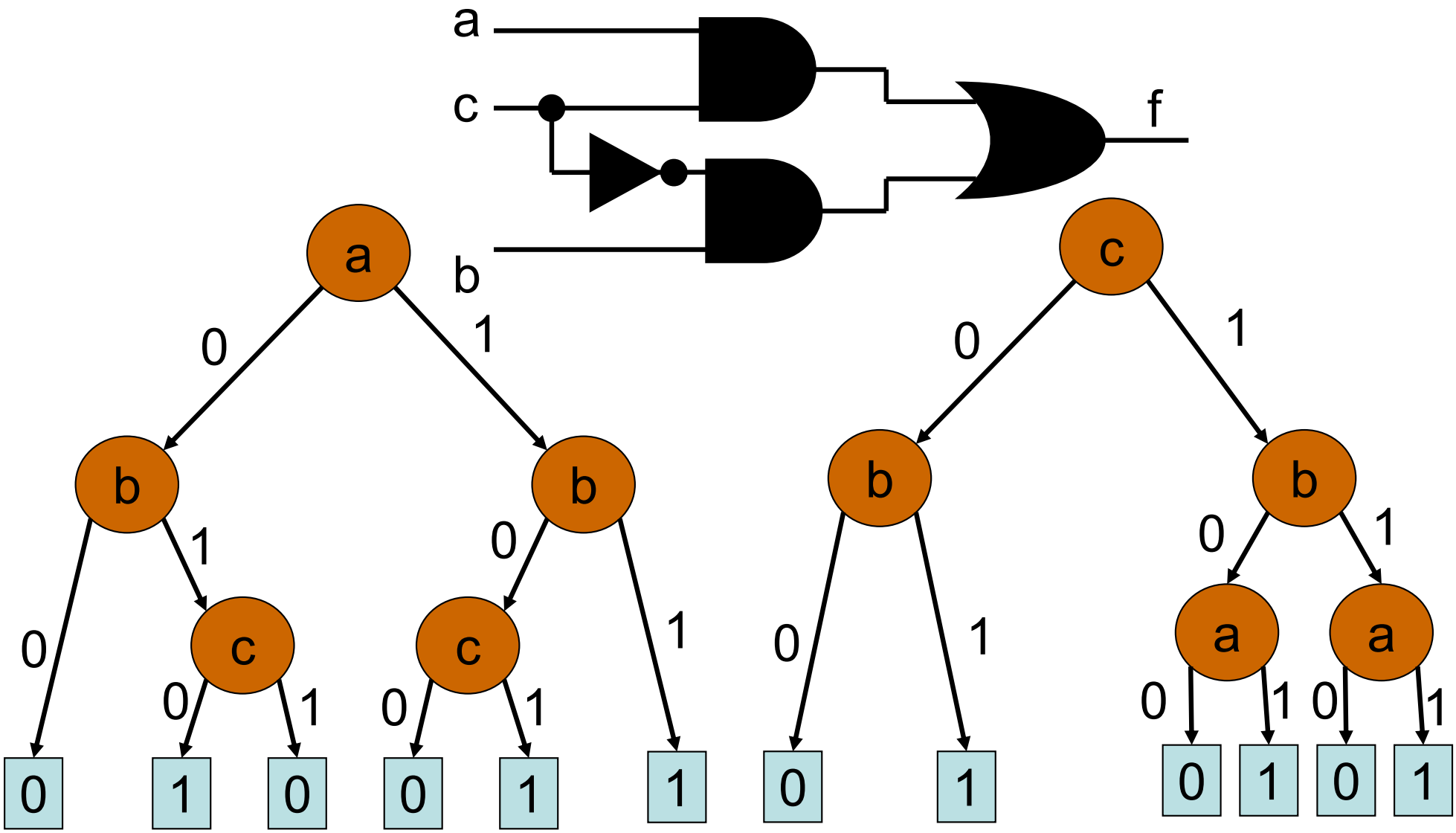


Decision Tree



Binary Decision Diagram

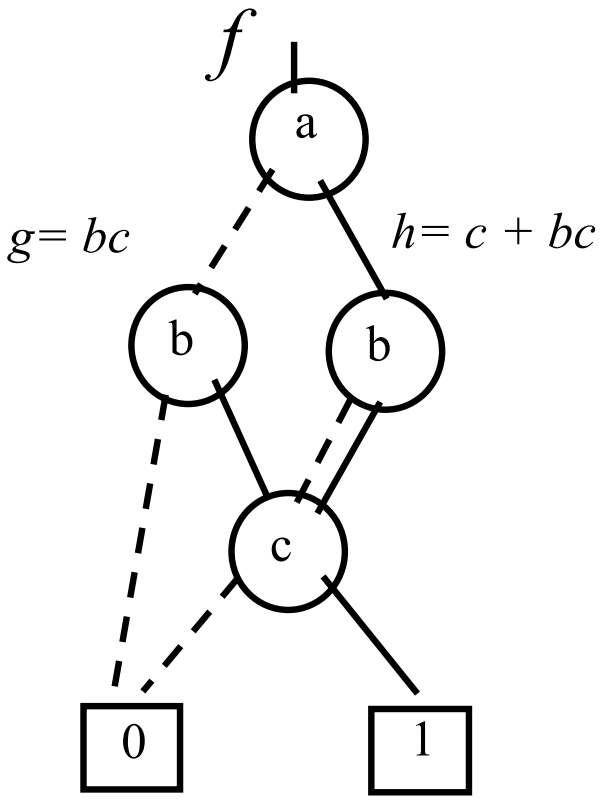
Binary Decision Diagrams With Different Input Ordering



Example of Binary Decision Diagram

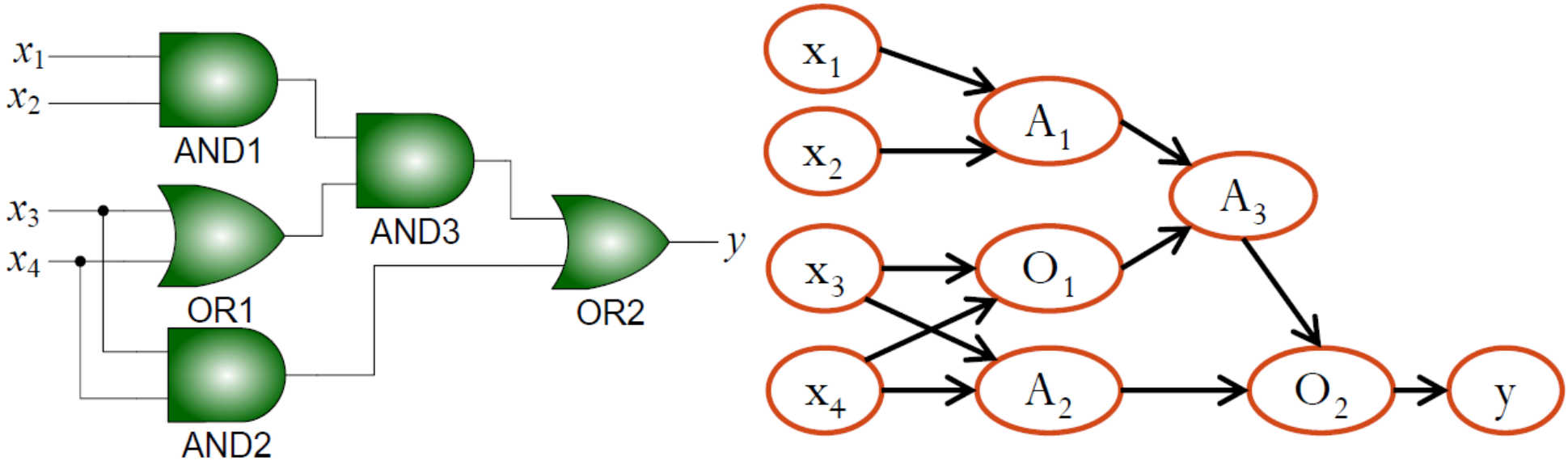
$$f = ac + bc$$

- $f_{\bar{a}} = f(a=0) = bc$
- $f_a = f(a=1) = c + bc$
- $g_{\bar{b}} = (bc)_{|b=0} = 0$
- $g_b = (bc)_{|b=1} = c$
- $h_{\bar{b}} = (c+bc)_{|b=0} = c$
- $h_b = (c+bc)_{|b=1} = c$



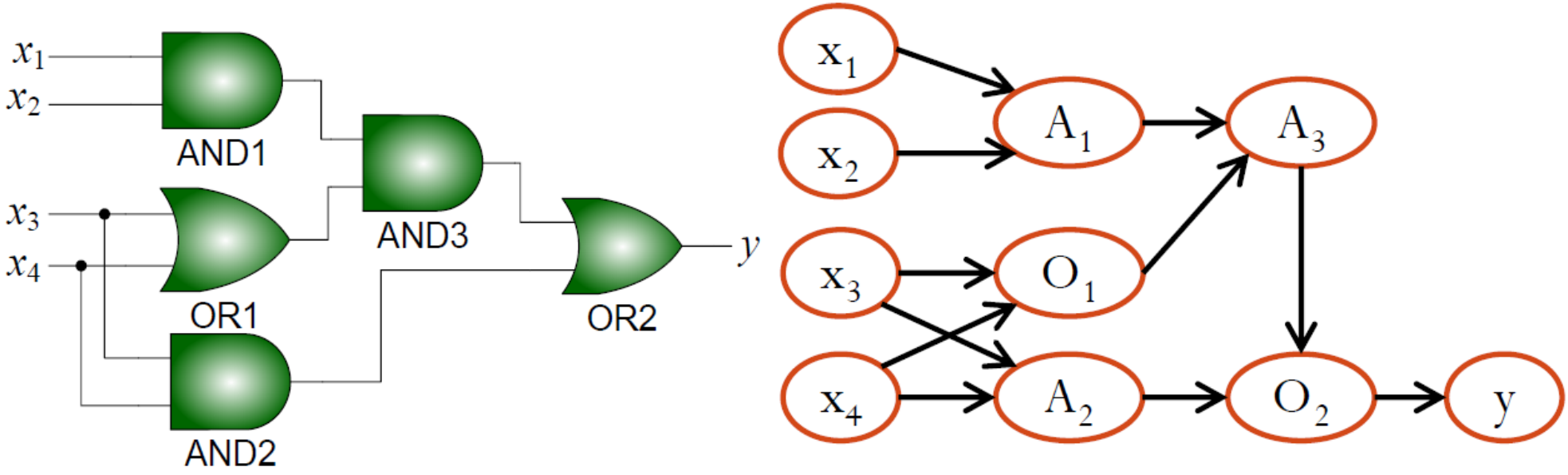
Combinational Logic Flow Diagram

- Represented as a directed graph.
  - Inputs, outputs, and gates ➔ nodes
  - Wires ➔ directed edges.
    - Why directed edges? Signal flow has direction.



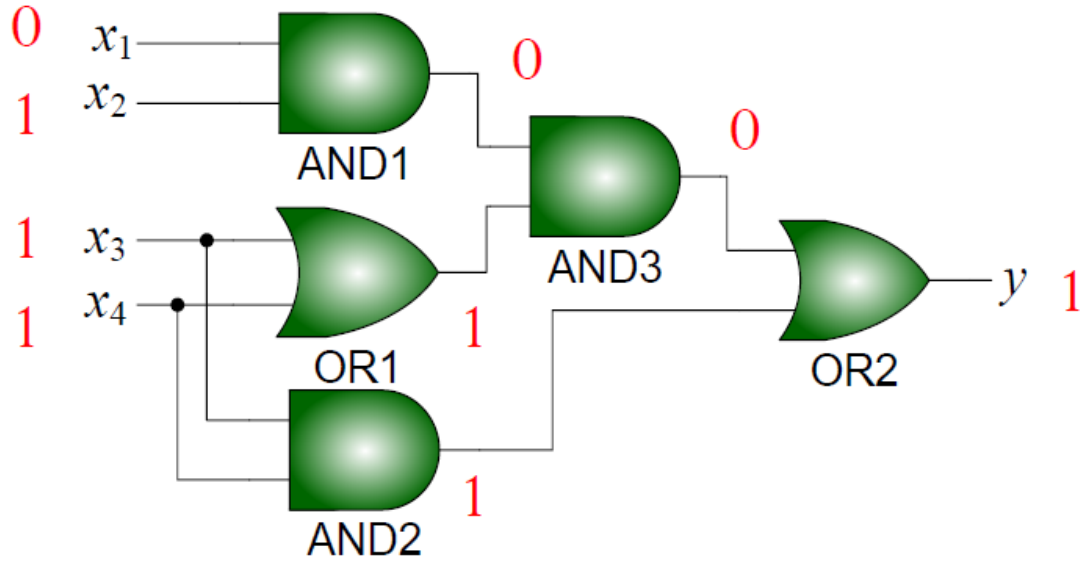
Combinational Logic Flow Diagram

- Since a combinational circuit has no loops, the corresponding graph is a **directed acyclic graphs (DAG)**.
  - DAG: A directed graph with no cycles.



Combinational Logic Flow Diagram

- Many operations on combinational circuit need to traverse it.
- Example: obtain the output given an input pattern.

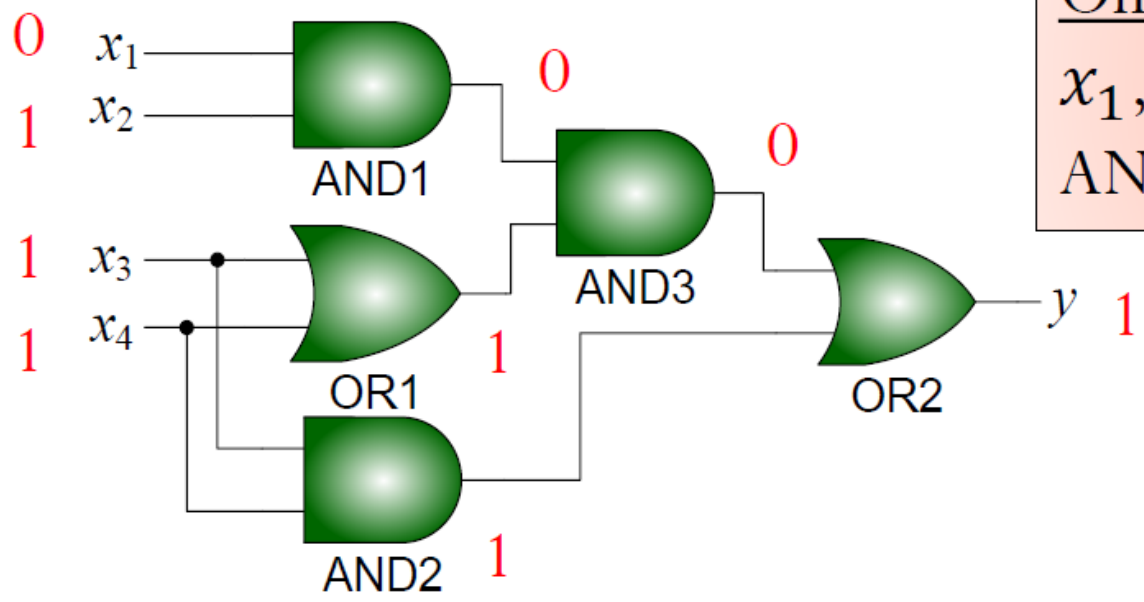


How to approach this kind of traversal as a computation?

Answer: Topological Sorting.

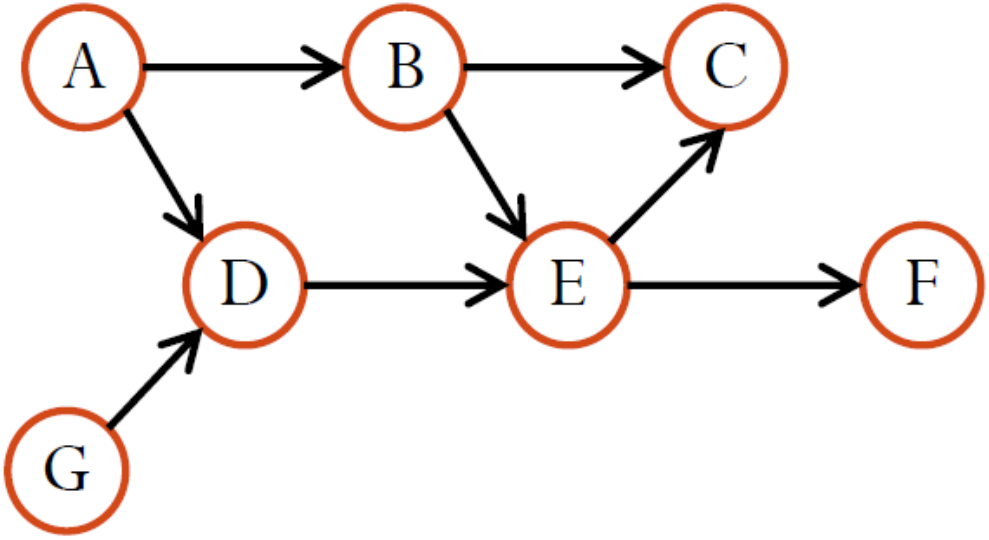
Combinational Logic Flow Diagram

- Observation on computing the output of a combinational circuit:
  - The output of each gate can be computed only when all of its input are known. ➔ We have to obtain its inputs before obtaining output.
  - In other words, if there is a wire (edge) from gate (node) u to v, then gate (node) u should be visited before gate (node) v.
    - This is exactly **topological sorting**.



One possible visiting order:
x_1, x_2, x_3, x_4, AND1, OR1, AND2, AND3, OR2, y.

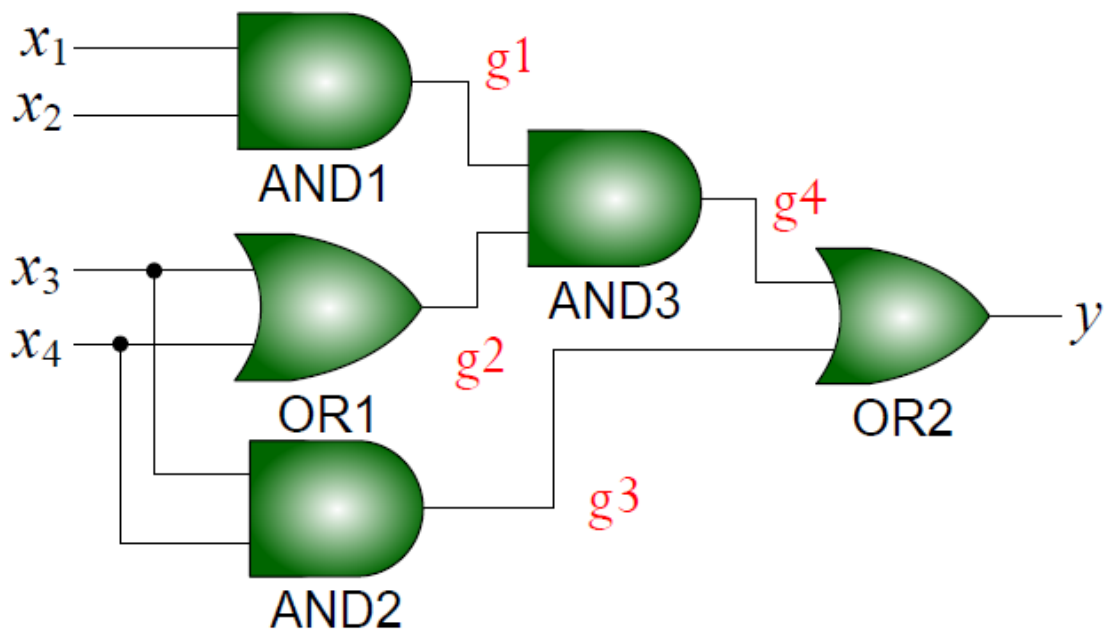
Combinational Logic Flow Diagram



- Topological sorting is not necessarily unique:
  - A, G, D, B, E, C, F and A, B, G, D, E, F, C are both topological sorting.
- Are the following orderings topological sorting?
  - A, B, E, G, D, C, F
  - A, G, B, D, E, F, C

Hardware Description Language Program

```
INPUT(x1)
INPUT(x2)
INPUT(x3)
INPUT(x4)
OUTPUT(y)
g1=AND(x1,x2)
g2=OR(x3,x4)
g3=AND(x3,x4)
g4=AND(g1,g2)
y=OR(g3,g4)
```



Outline

- **Basics of Combinational Circuits**
  - Boolean Algebra
  - Truth Table & Logic Gates
- **Canonical Implementation**
  - Sum-of-Products (SOP)
  - Product-of-Sum (POS)
- **Circuit Simplification**
  - Karnaugh Map
  - Half Adder and Full Adder
- **Designing Large Logic Circuits**
  - Shannon's Expansion Theorem
  - Topological ordering and Hardware Description Program
- **Logic Games**
  - Logic Diagrams and Puzzles

Background Reading: Chapters 4 and 5 in Textbook

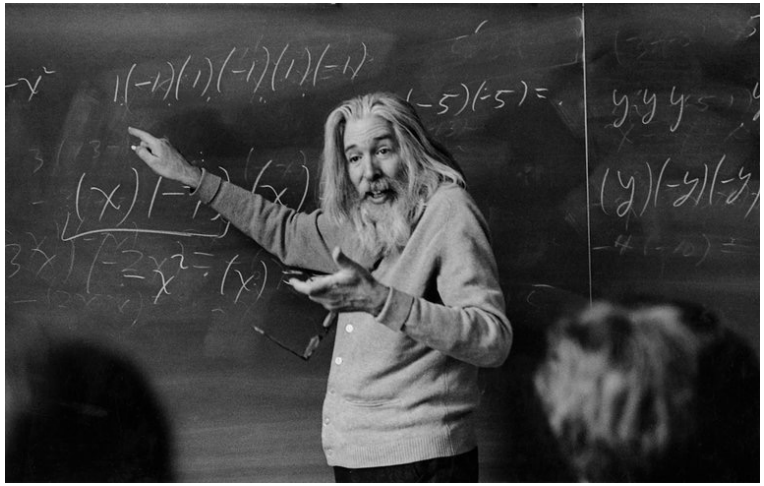
Digital Logic Puzzle (Classic Example)

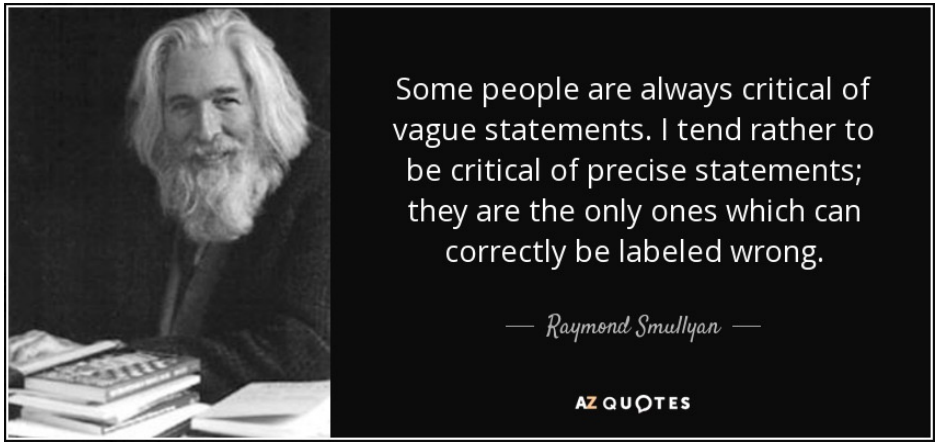
- A very special island is inhabited only by knights and knaves. Knights always tell the truth, and knaves always lie.

  You meet two inhabitants: Zoey and Mel.

  - Zoey tells you that Mel is a knave.
  - Mel says, ‘Neither Zoey nor I are knaves.’

  So who is a knight and who is a knave?



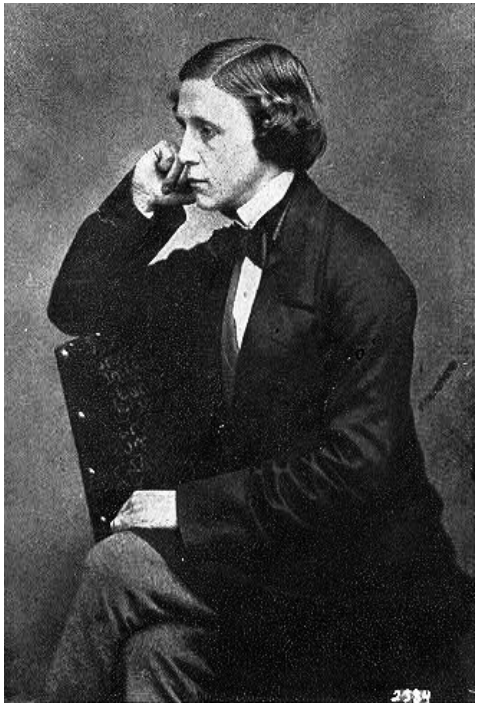


Digital Logic Puzzle (Textbook Example)

- **On the table before you are three small boxes labeled A, B and C.**
- **Inside each box is a colored plastic chip. One chip is red, one is white and one is blue.**
- **You do not know which chip is in which box.**
- **Then you are told about the following three statements, however exactly only one is true (you do not know which one).**
- **(a) Box A contains the red chip.**
- **(b) Box B does not contain the red chip.**
- **(c) Box C does not contain the blue chip.**

- **Determine the color of the chip in each box.**

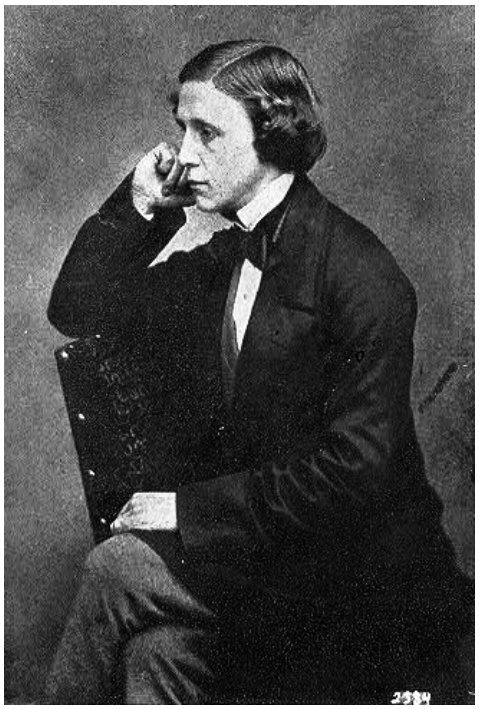
Lewis Carroll & Game of Logic

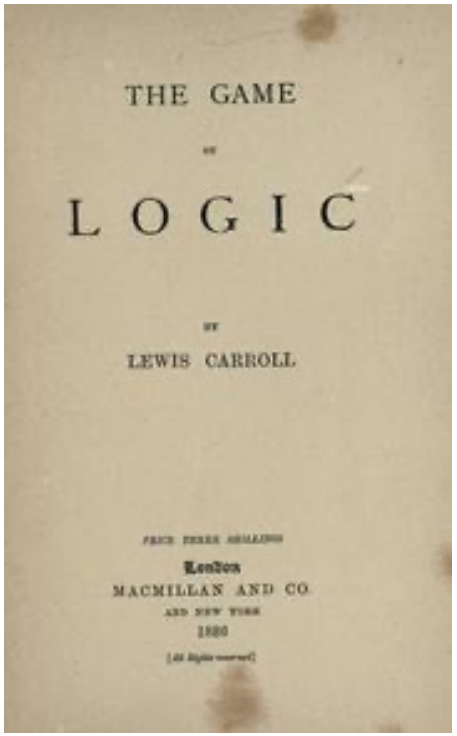




- Lewis Carroll
- Alice's Adventures in Wonderland

Lewis Carroll & Game of Logic





If these three sentences are true, what can you conclude?

1) No experienced person is incompetent.

2) Jenkins is always blundering.

3) No competent person is always blundering.

- Lewis Carroll
- Carroll's diagram is a forerunner of the Karnaugh map
- Visualization of *symbolic logic* vs. *digital logic*
- Venn's diagram, Carroll's diagram, Truth-table, Acyclic graphs etc are important mathematical tools in algebra

Games with Binary Number System

Nim

Two players take turns removing objects from distinct heaps. On each turn, a player must remove at least one object, and may remove any number of objects provided they all come from the same heap.

The goal of the game is to be the player who removes the last object.

Let's Play!!

Games with Binary Number Svstem

The mathematics behind Nim

Patented Sept. 24, 1940 2,215,544

UNITED STATES PATENT OFFICE

2,215,544

MACHINE TO PLAY GAME OF NIM

Edward U. Condon, Edgewood, and Gereld L. Tawney and Willard A. Derr, Wilkinsburg, Pa., assignors to Westinghouse Electric & Manufacturing Company, East Pittsburgh, Pa., a corporation of Pennsylvania

Application April 26, 1940, Serial No. 331,784

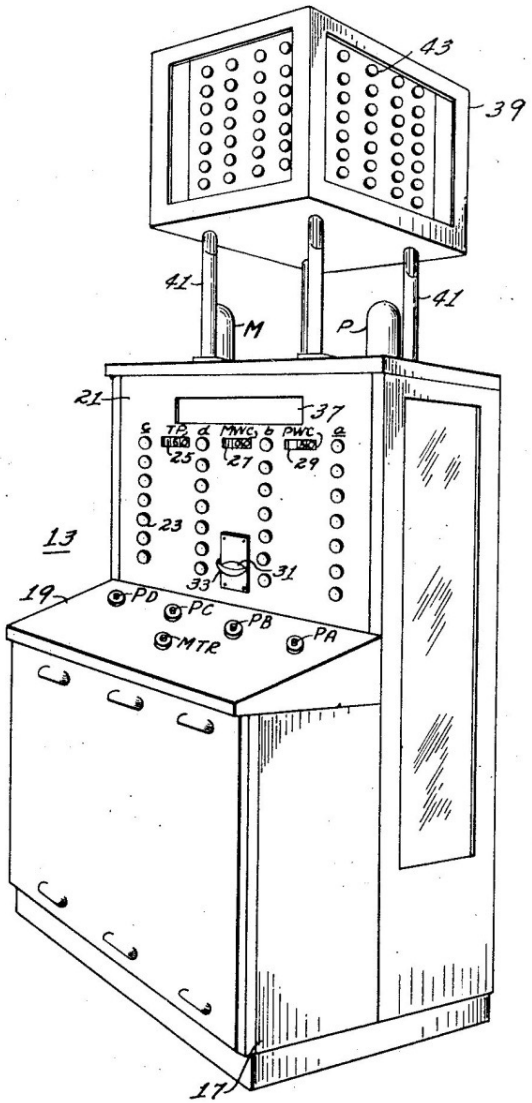
17 Claims. (Cl. 273—130)

Our invention relates to control apparatus and has particular relation to electrical apparatus for

Initially the array of sets may be analyzed as follows:

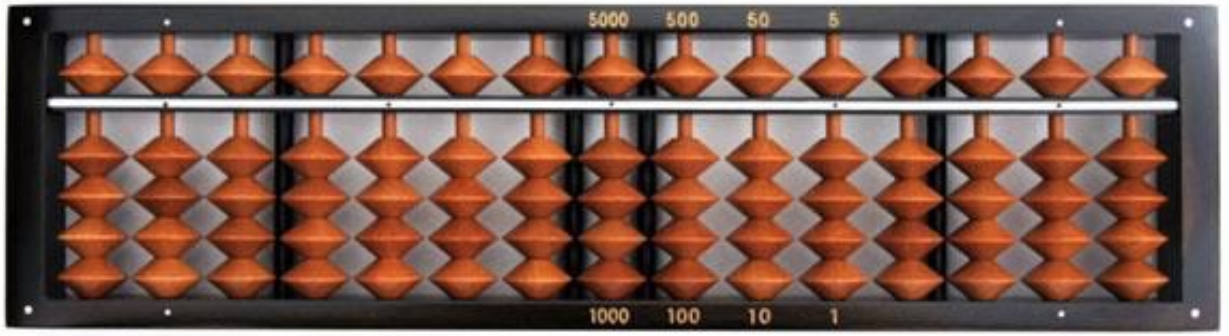
For a better understanding of Nim and the manner in which it is played, we may consider an example. Assume that originally there are three sets of like elements; 9 elements in set No. 1; 7 elements in set No. 2, and 5 elements in set No. 3. In playing the game, each player, in his turn, may remove any number of elements from set No. 1, set No. 2, or set No. 3. He may, for example, remove 6 elements from set No. 1 in making his first play and after the other player moves, he may remove any number of elements from the same or another set, say set No. 3. The player removing the last element is the winner.





Computing Machines









Extension

- **Symmetric Boolean Functions**
- **The distribution of 1s and 0s in K-map for symmetric functions? The alternate 1s and 0s for XOR**
- **N-bit full adder in cascade is majority voting in pairs of N users, the "overflow bit" denotes the outcome**

  If there is odd number of users, i.e., N is odd, then the "odd person's" vote is input at the righthand-side full adder.
- **Half adder**
- **→ binary arithmetic**
- **→ Galois field GF(2) for even-odd (addition is XOR, mult is AND)**
- **→ majority voting, parity checking**
- **→ Universal gates**

Shannon's Expansion Theorem

- What ***else*** can you do with cofactors?
- Suppose you have 2 functions $F(X)$ and $G(X)$, where $X = (x_1, x_2, \ldots, x_n)$.
- Suppose you make a new function H, from F and G, say…
  - $H = \bar{F}$
  - $H = F \cdot G$, i.e., $H(X) = F(X) \cdot G(X)$
  - $H = F + G$, i.e., $H(X) = F(X) + G(X)$
  - $H = F \oplus G$, i.e., $H(X) = F(X) \oplus G(X)$
- Question: can you tell anything about H's cofactors from those of F and G?
  - $(F \cdot G)_x =$ **what**? $(\bar{F})_x =$ **what**?

Shannon's Expansion Theorem

- Cofactors of F and G tell you everything you need to know.
- Complements
  - $(\bar{F})_x = \overline{(F_x)}$
  - In English: cofactor of complement is complement of cofactor.
- Binary Boolean operators
  - $(F \cdot G)_x = F_x \cdot G_x$ cofactor of AND is AND of cofactors
  - $(F + G)_x = F_x + G_x$ cofactor of OR is OR of cofactors
  - $(F \oplus G)_x = F_x \oplus G_x$ cofactor of XOR is XOR of cofactors
- **Very useful**! Can often help in getting cofactors of complex formulas.

Shannon's Expansion Theorem

- So, do Boolean functions have "**derivatives**"?
  - Actually, yes. Trick is how to define them…

- Basic idea
  - For real-valued $f(x)$, $\frac{df}{dx}$ tells how f changes when x changes.
  - For 0,1-valued Boolean function, we cannot change x by small delta.
    - Can only change 0 $\leftrightarrow$ 1, but can still ask how f changes with x …
  - For Boolean function $f(x)$, define

$$\frac{\partial f}{\partial x} = f_x \oplus f_{\overline{x}}$$

Shannon's Expansion Theorem

$$\frac{\partial f}{\partial x} = f_x \oplus f_{\overline{x}}$$

- Compare value of f when $x = 0$ against when $x = 1$.
  - $\frac{\partial f}{\partial x} == 1$ if and only if $f(x = 0)$ is different from $f(x = 1)$.

- $\frac{\partial f}{\partial x}$ is also known as **Boolean difference**.

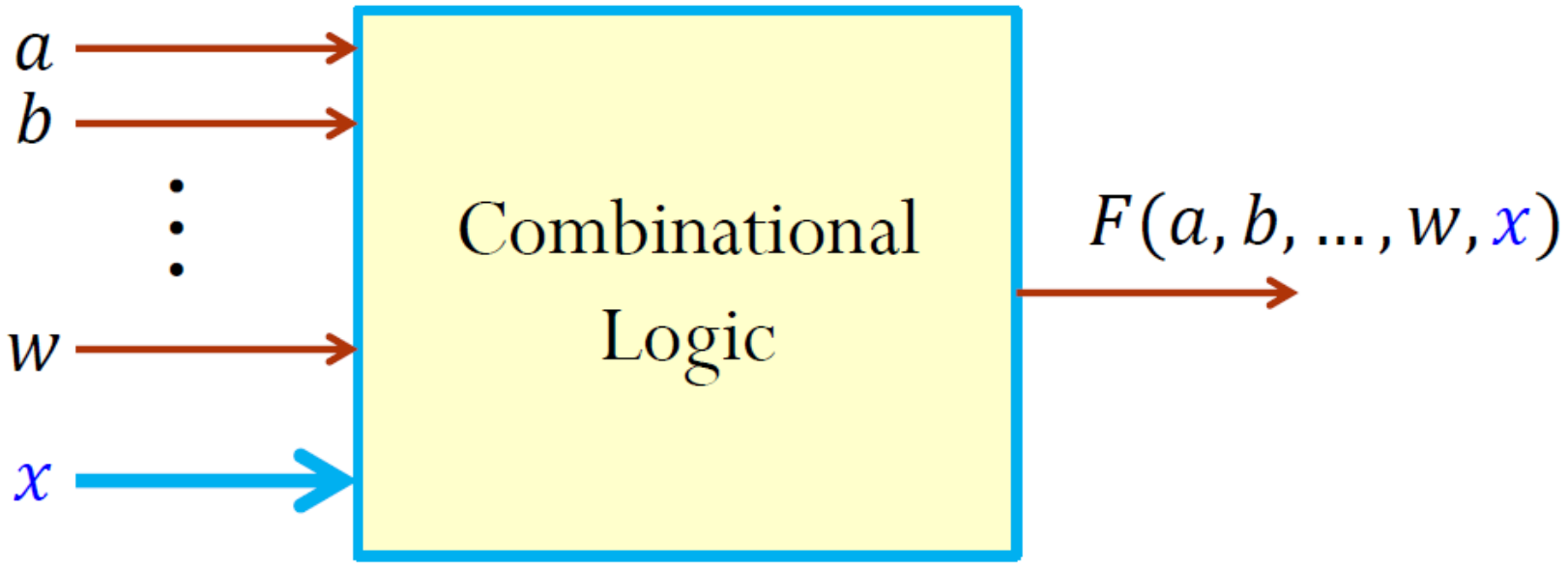
- If function f is constant ($f = 1$ or $f = 0$ for all inputs), then $\partial f / \partial x = 0$ for any x.

Shannon's Expansion Theorem

- Consider simple examples for $\partial f / \partial x$.
- Inverter: $f = \bar{x}$
  - $f_x = 0,\ f_{\bar{x}} = 1,\ \partial f / \partial x = f_x \oplus f_{\bar{x}} = 1$
- AND: $f = xy$
  - $f_x = y,\ f_{\bar{x}} = 0,\ \partial f / \partial x = f_x \oplus f_{\bar{x}} = y$
- OR: $f = x + y$
  - $f_x = 1,\ f_{\bar{x}} = y,\ \partial f / \partial x = f_x \oplus f_{\bar{x}} = \bar{y}$
- XOR: $f = x \oplus y$
  - $f_x = \bar{y},\ f_{\bar{x}} = y,\ \partial f / \partial x = f_x \oplus f_{\bar{x}} = 1$

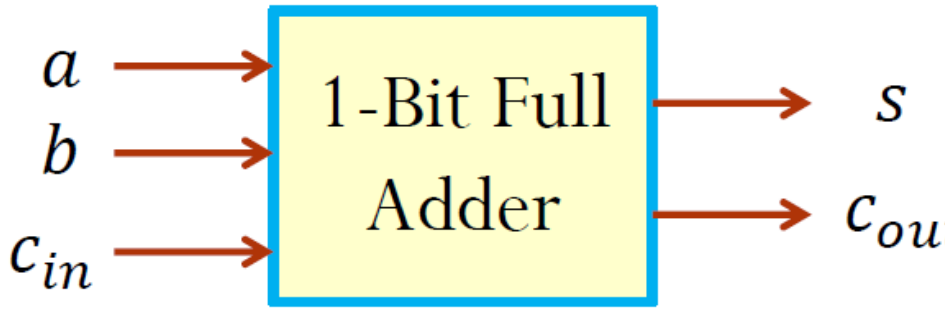
Meaning: When $\partial f / \partial x = 1$, then f changes if x changes!

Shannon's Expansion Theorem



- What does $\partial F(a, b, \ldots, w, x)/\partial x = 1$ mean?
  - If you apply a pattern of inputs $(a, b, \ldots, w)$ that makes $\partial F/\partial x = 1$, then any change in x will force a change in output F.

Shannon's Expansion Theorem



$$s = a \oplus b \oplus c_{in}$$

$$c_{out} = ab + ac_{in} + bc_{in}$$

- What is $\partial c_{out}/\partial c_{in} = 1$?
  - $c_{out}(c_{in} = 1) = a + b$
  - $c_{out}(c_{in} = 0) = ab$
  - $\partial c_{out}/\partial c_{in} = c_{out}(c_{in} = 1) \oplus c_{out}(c_{in} = 0)$
    $= (a + b) \oplus (ab) = a \oplus b$
- Make sense?
  - $a \oplus b = 1 \Longrightarrow a \neq b$